



Chapter 10

セッションとクッキー

アンケートフォームや買い物サイトなどのように、複数の Web ページを移動しながら入力データを集めていく Web サービスでは、セッションやクッキーの機能の活用が欠かせません。セッションやクッキーの使い方をその注意点と合わせて解説します。

Section 10-1 セッション処理の基礎

Section 10-2 フォーム入力の値をセッション変数に入れる

Section 10-3 複数ページでセッション変数を利用する

Section 10-4 クッキーを使う

Section 10-5 クッキーで訪問カウンタを作る

Section 10-6 複数の値を 1 つにまとめてクッキーに保存する

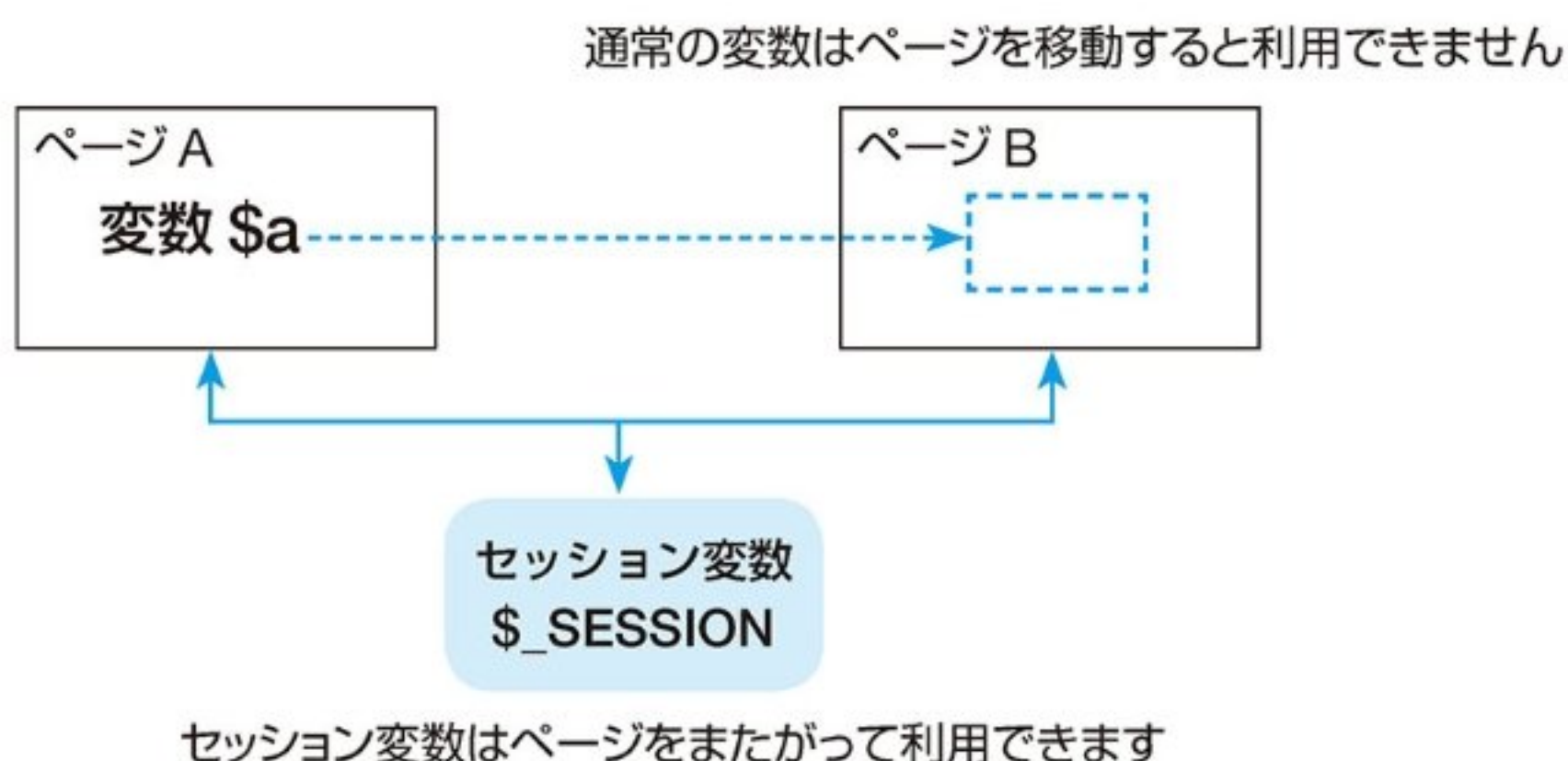
Section 10-1

セッション処理の基礎

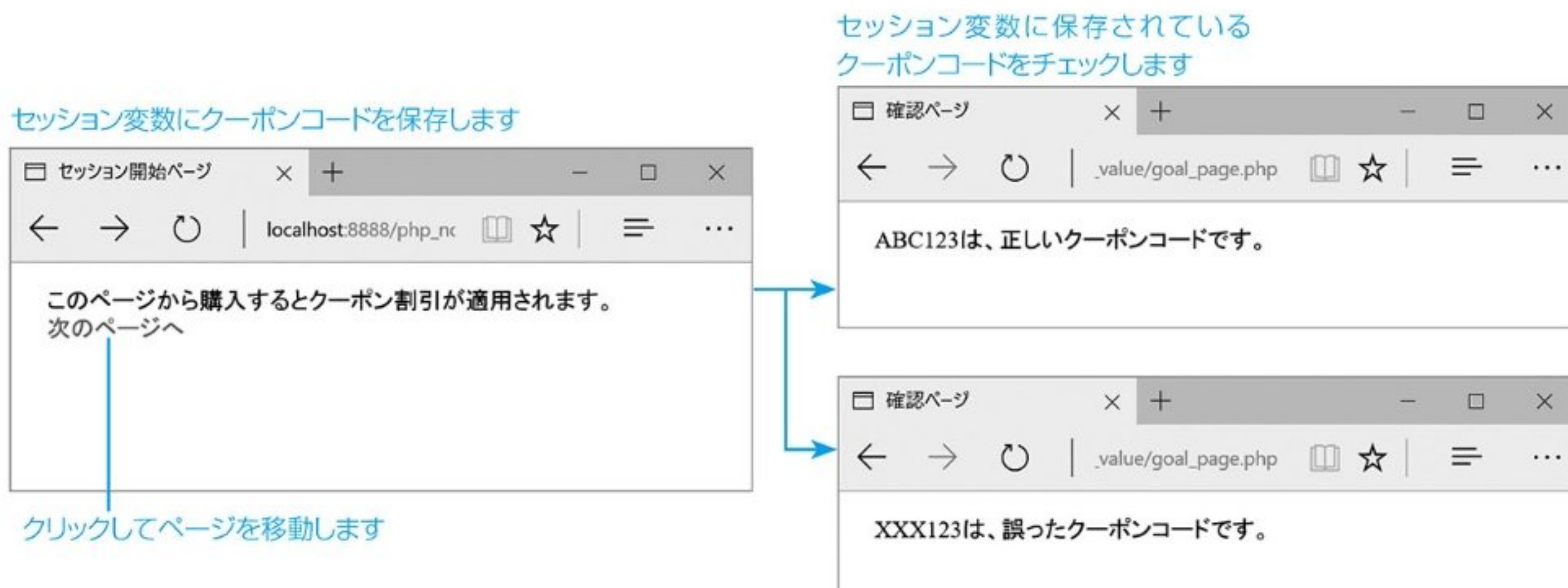
セッションの機能を利用すると、複数の Web ページで共通して使えるセッション変数を利用できるようになります。この節では変数の値を次のページに渡すだけの簡単な例でセッションの基礎を説明します。

セッションの概要

Web ページを移動してしまうと前のページで利用していた変数の値は使えなくなります。セッションの機能を利用することで、複数の Web ページで共通して使えるセッション変数 `$_SESSION` を利用できるようになります。

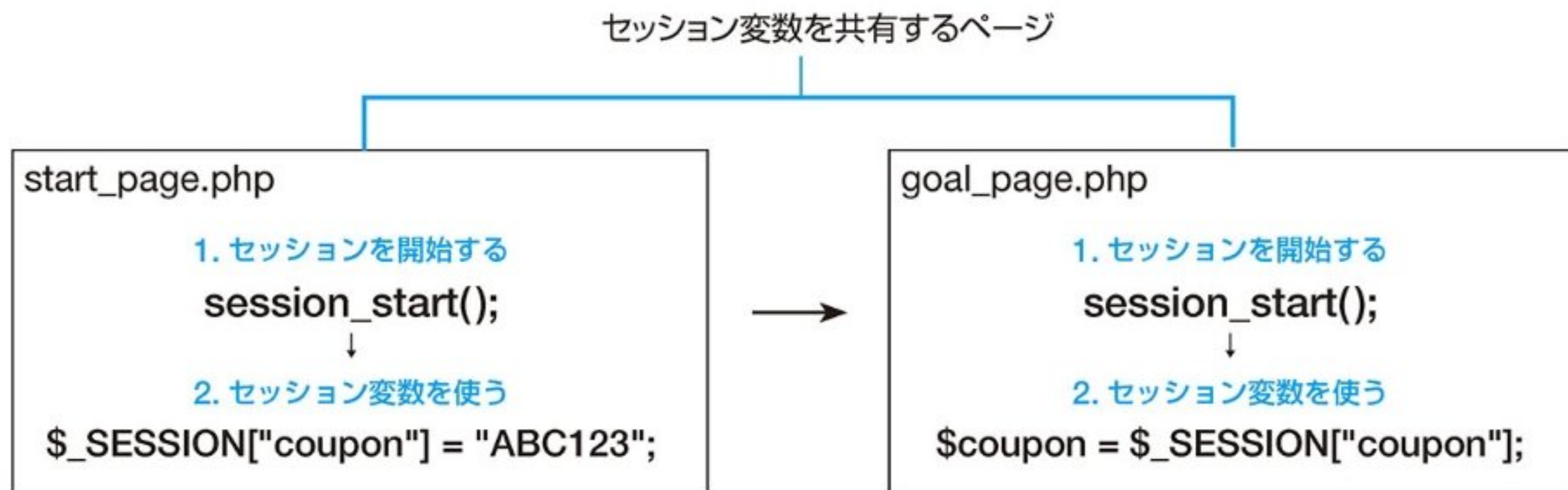


では、実際にどのようにセッションを使っていくのかを簡単な例で示します。次の例ではクーポンコードをセッション変数に代入し、クリックで開いたページで正しいクーポンコードを受け取ったかどうかを判定しています。ページの移動には `<a>` タグのリンクを使っています。



セッション変数に値を保存する

セッション変数を利用するには、各ページでセッションを開始します。セッションを開始するとスーパーグローバル変数であるセッション変数 `$_SESSION` を利用できるようになります。



php セッション変数に値を保存する

«sample» session_value/start_page.php

```

01:  <?php
02:  // セッションの開始
03:  session_start();  ———— セッションを開始します。
04:  ?>               これより前に空白行や HTML コードがあってははいけません
05:
06:  <!DOCTYPE html>
07:  <html lang="ja">
08:  <head>
09:  <meta charset="utf-8">
10:  <title> セッション開始ページ </title>
11:  <link href="../../../css/style.css" rel="stylesheet">
12:  </head>
13:  <body>
14:  <div>
15:      このページから購入するとクーポン割引が適用されます。<br>
16:      <?php
17:          // セッション変数に値を代入する
18:          $_SESSION["coupon"] = "ABC123";  ———— セッション変数を使います
19:      ?>
20:      <a href="goal_page.php"> 次のページへ </a>
21:  </div>
22:  </body>
23:  </html>
  
```

Part 3

Chapter

8

Chapter

9

Chapter

10

Chapter

11

セッションを開始する

セッションは `session_start()` で開始します。このとき、`session_start()` より前に空白や改行を含めて HTML コードがあってははいけません。PHP のコメントを含めて、PHP コードは前にあっても構いません。したがって、サンプルのようにコードの最初で `session_start()` を実行します。

php セッションの開始

«sample» session_value/start_page.php

```
03: session_start();
```

セッション変数に値を保存する

セッションを開始するとスーパーグローバル変数の `$_SESSION` を利用できるようになります。`$_SESSION` は `$_POST` と同じく連想配列です（連想配列 ➡ P.202）。ここでは `"coupon"` をキーにして `"ABC123"` を値として保存しています。

php セッション変数に値を代入する

«sample» session_value/start_page.php

```
18: $_SESSION["coupon"] = "ABC123";
```

移動したページでセッション変数の値を取り出す

移動したページでセッション変数の値を取り出す手順は、基本的に値を保存した場合と同じです。この例ではセッション変数に保存したクーポンコードが正しいコードかどうかを確認しています。

php セッション変数で受け取ったクーポンコードが正しいかどうか調べる

«sample» session_value/goal_page.php

```
01: <?php
02: require_once("../lib/util.php");
03: // セッションの開始
04: session_start(); —— セッションを開始します
05: ?>
06:
07: <!DOCTYPE html>
08: <html lang="ja">
09: <head>
10:   <meta charset="utf-8">
11:   <title> 確認ページ </title>
12:   <link href="../css/style.css" rel="stylesheet">
13: </head>
14: <body>
15: <div>
16:   <?php
17:   // セッション変数を調べる
18:   if(isset($_SESSION["coupon"])){
19:     // クーポンコードを取り出す
20:     $coupon = $_SESSION["coupon"]; —— セッション変数からクーポンコードを取り出します
21:     // 正しいクーポンコード
22:     $couponList = ["ABC123", "XYZ999"];
```

```

23:         // クーポンコードをチェックする
24:         if (in_array($coupon, $couponList)){
25:             echo es($coupon), " は、正しいクーポンコードです。";
26:         } else {
27:             echo es($coupon), " は、誤ったクーポンコードです。";
28:         }
29:     } else {
30:         echo " セッションエラーです ";
31:     }
32:     ?>
33: </div>
34: </body>
35: </html>

```

セッションを開始する

移動した先のページでもセッションの機能を利用するには、まず、セッションを開始します。ここでは、セッションを開始する `session_start()` より先に `require_once()` で `util.php` を読み込んでいます。

php セッションを開始する

«sample» session_value/goal_page.php

```

01: <?php
02: require_once("../lib/util.php");
03: // セッションの開始
04: session_start();
05: ?>

```

読み込むファイルに空白行や HTML コードが含まれているとエラーになります

NOTE

session_start() を実行すると Warning が出る

`session_start()` よりも前に空白や改行なども含めて HTML コードがあるとユーザの環境によっては警告が出力されます。出力されるエラーメッセージは次のようなものです。このエラーは「Warning」なので、警告は出ますが処理は中断せずにそのまま続くコードが実行されます。

Warning: session_start(): Cannot send session cookie - headers already sent

Warning: session_start(): Cannot send session cache limiter - headers already sent

`session_start()` よりも前に PHP コードがあるのは問題ありません。ただし、この節のサンプルのように `session_start()` よりも前にほかのファイルを読み込む場合は注意が必要です。もし、読み込んだファイル内に空白行などが含まれているとユーザの環境によってはエラーになります。

この問題を解決するには `session_start()` を 1 行目で実行すればよいのですが、根本的な解決として読み込む php ファイルの中身を確認して空白行や HTML コードを取り除いておきましょう。複数の `<?php ~ ?>` が含まれているコードの場合、`<?php ~ ?>` と `<?php ~ ?>` の間に空白行などがないかをチェックしてみてください。echo で文字列を出力している場合も同様のエラーになるので特に注意が必要です。読み込む PHP ファイルの最後の閉じたタグ `?>` を省略すればタグは自動で閉じますが、`?>` が書いてある場合は、その後の空白行を見逃さないでください。

セッション変数の値を取り出す

セッション変数\$_SESSIONからの値の取り出し方は\$_POSTから値を取り出す場合と同じ方法です。まず、保存したキーに値がセットされているかどうかを `isset($_SESSION["coupon"])` でチェックし、セットされていたならば `$_SESSION["coupon"]` の値を取り出します。値がセットされていなかったならば、なんらかの理由でセッションエラーになります。

php セッション変数の値を取り出す

«sample» session_value/goal_page.php

```
17: // セッション変数を調べる
18: if(isset($_SESSION["coupon"])){ ــــــــــــــــ coupon キーに値がセットされているかチェックします
19:     // クーポンコードを取り出す
20:     $coupon = $_SESSION["coupon"];
21:     . . .
29: } else {
30:     echo "セッションエラーです ";
31: }
```

クーポンコードをチェックして表示する

このサンプルでは取り出したクーポンコードが正しいコードかどうかをチェックします。正しいコードが配列 `$couponList` に入っているとき、セッション変数から取り出した `$coupon` の値が `$couponList` の値にあるかどうかを `in_array()` を使ってチェックします。

チェックした結果が正しいクーポンコードであれば問題ありませんが、誤ったクーポンコードをそのままブラウザに表示するのは危険です。セッション変数の値をブラウザに表示する際には、習慣として必ず HTML エスケープを行うようにしましょう。ここでは最初に読み込んだ `util.php` で定義してある `es()` を使って HTML エスケープしています (P.306)。

php クーポンコードをチェックする

«sample» session_value/goal_page.php

```
21: // 正しいクーポンコード
22: $couponList = ["ABC123", "XYZ999"];
23: // クーポンコードをチェックする
24: if (in_array($coupon, $couponList)){ ــــــــــــــــ 値が配列に含まれているかチェックします
25:     echo es($coupon), " は、正しいクーポンコードです。 ";
26: } else {
27:     echo es($coupon), " は、誤ったクーポンコードです。 "; ــــــــــــــــ HTML エスケープして表示します
28: }
```

Section 10-2

フォーム入力値をセッション変数に入れる

セッションの利用では複数ページにわたるフォーム入力の回答を蓄えていくという使い方があります。この節ではセッションのそのような使い方の最初の例として、フォーム入力値が1ページだけのものを作ります。セッションを破棄する方法も説明します。

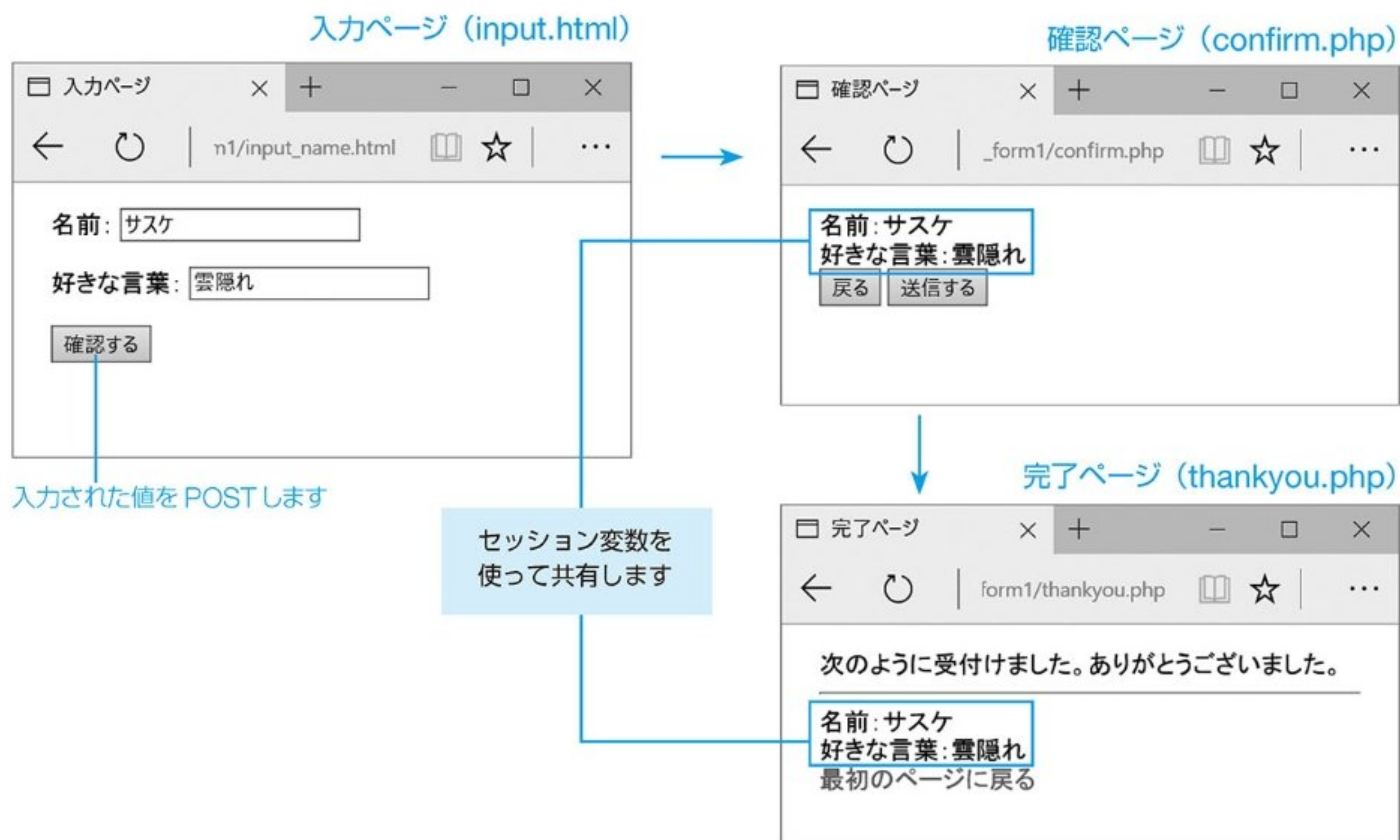
POSTされた値をセッション変数で受け継ぐ

フォーム入力からPOSTされた値を複数のページで利用するには、`$_POST`の値を`$_SESSION`の値として代入して使っていきます。

次に説明する例では図に示すように最初の入力ページ (`input.html`) で「名前」と「好きな言葉」をフォーム入力します。PHP コードは含まれていないのでHTML ファイルとして保存してあります。

名前と好きな言葉を入力して「確認する」ボタンをクリックするとPOSTされて確認ページ (`confirm.php`) が開きます。確認ページではフォーム入力された内容を表示します。訂正があれば「戻る」ボタンで入力ページに戻り、このままでよければ「送信する」ボタンで完了ページ (`thankyou.php`) へと進みます。

ここまでならば確認ページに`$_POST`変数の値を表示するだけで済みますが、完了ページでも名前と好きな言葉を表示します。確認ページから完了ページに値を送るには`<input>`タグの`hidden`属性を使う方法もありますが (P.327)、ここでセッションを活用します。



名前と好きな言葉を入力する入力ページを作る

入力ページを作る input.html には、「名前」と「好きな言葉」を入力するフォームがあります。どちらも `<input>` タグで作り、POST で送信する際の name 属性に「名前」は "name"、「好きな言葉」は "kotoba" が設定してあります。「名前」には placeholder 属性で " ニックネーム可 " のプレースホルダが付けてあります。どちらも初期値の設定がないので、確認ページから戻ってきた場合も空の状態から始まります。

html 入力ページを作る

«sample» session_form1/input.html

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>入力ページ</title>
06: <link href="../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:   <form method="POST" action="confirm.php">
11:     <li><label>名前:
12:       <input type="text" name="name" placeholder=" ニックネーム可 ";>
13:     </label></li>
14:     <li><label>好きな言葉:
15:       <input type="text" name="kotoba";>
16:     </label></li>
17:     <li><input type="submit" value=" 確認する "></li>
18:   </ul>
19: </form>
20: </div>
21: </body>
22: </html>

```

name と kotoba の値を confirm.php に POST します

名前フィールド

言葉フィールド

送信ボタン

POST された値をチェックする確認ページを作る

入力ページの「確認する」ボタンをクリックすると確認ページが開きます。確認ページでは次の3つのことを行います。

1. POST された値をセッション変数に移す
2. 値をチェックして表示する
3. 「戻る」、「送信する」のフォームボタンを作る

php 確認ページを作る

«sample» session_form1/confirm.php

```

01: <?php
02: require_once("../lib/util.php");
03: // セッションの開始
04: session_start(); // セッションを開始します
05: ?>
06:
07: <?php
08: // 文字エンコードの検証
09: if (!cken($_POST)){
10:     $encoding = mb_internal_encoding();
11:     $err = "Encoding Error! The expected encoding is " . $encoding ;
12:     // エラーメッセージを出して、以下のコードをすべてキャンセルする
13:     exit($err);
14: }
15: ?>
16:
17: <?php
18: // POSTされた値をセッション変数に受け渡す
19: if (isset($_POST['name'])){
20:     $_SESSION['name'] = trim(mb_convert_kana($_POST['name'], "s"));
21: }
22: if (isset($_POST['kotoba'])){
23:     $_SESSION['kotoba'] = trim(mb_convert_kana($_POST['kotoba'], "s"));
24: }
25: // 入力データの取り出しとチェック
26: $error = [];
27: // 名前
28: if (empty($_SESSION['name'])){
29:     // 未設定のときエラー
30:     $error[] = "名前を入力してください。";
31: } else {
32:     // 名前を取り出す
33:     $name = $_SESSION['name'];
34: }
35: // 好きな言葉
36: if (empty($_SESSION['kotoba'])){
37:     // 未設定のときエラー
38:     $error[] = "好きな言葉を入力してください。";
39: } else {
40:     // 好きな言葉を取り出す
41:     $kotoba = $_SESSION['kotoba'];
42: }
43: ?>
44:
45: <!DOCTYPE html>
46: <html lang="ja">
47: <head>
48:     <meta charset="utf-8">
49:     <title>確認ページ</title>
50:     <link href="../css/style.css" rel="stylesheet">
51: </head>
52: <body>
53: <div>
54:     <form>
55:     <?php if (count($error)>0){ ?>
56:     <!-- エラーがあったとき -->
57:     <span class="error"><?php echo implode('<br>', $error); ?></span><br>

```

POSTされた「名前」をセッション変数に代入します

POSTされた「好きな言葉」をセッション変数に代入します

if 文で分岐します

\$errorに入っているエラー文を表示します

Part 3

Chapter

8

Chapter

9

Chapter

10

Chapter

11

```

58:     <span>
59:         <input type="button" value=" 戻る " onclick="location.href='input.html'">
60:     </span>
61:     <?php } else { ?>
62:     <!-- エラーがなかったとき -->
63:     <span>
64:         名前:<?php echo es($name); ?><br>
65:         好きな言葉:<?php echo es($kotoba); ?><br>
66:         <input type="button" value=" 戻る " onclick="location.href='input.html'">
67:         <input type="button" value=" 送信する " onclick="location.href='thankyou.php'">
68:     </span>
69:     <?php } ?>
70: </form>
71: </div>
72: </body>
73: </html>

```

POST された名前と言葉を表示します

クリックで thankyou.php へ移動します

POST された値をセッション変数に移す

まず最初にセッション変数を利用するために、`session_start()` を実行してセッションを開始します。

php セッションを開始する

«sample» session_form1/confirm.php

```
04: session_start();
```

次に `$_POST['name']` に値があればその値をセッション変数の `$_SESSION['name']` に代入します。同様に `$_POST['kotoba']` に値があれば `$_SESSION['kotoba']` に値を代入します。セッション変数に移す理由は、完了ページでこれらの値を使うからです。isset() で値が設定されているかどうかだけを確認したならば、全角空白を半角空白に変換した後で trim() で前後の不要な空白やタブを取り除いた値をセッション変数に移します (trim() 参照 P.161)。

php POST された値をセッション変数に移す

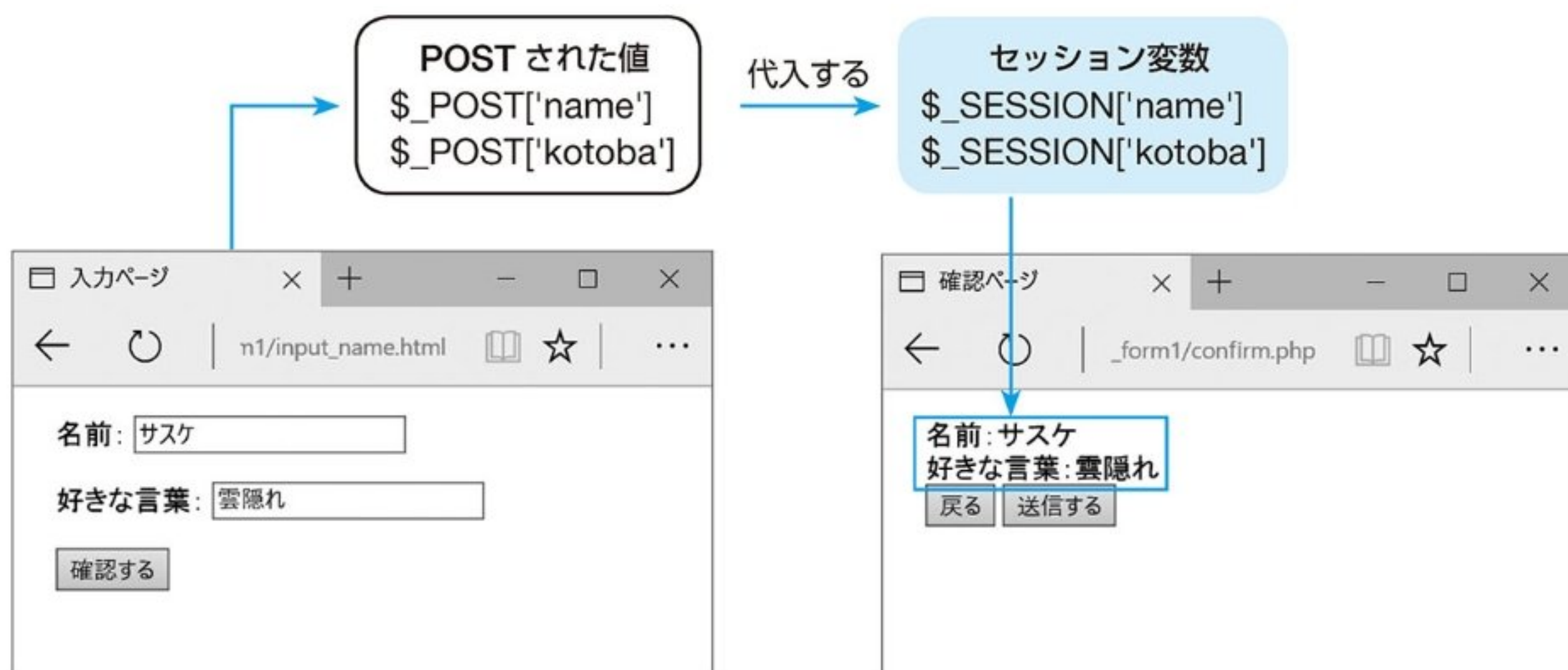
«sample» session_form1/confirm.php

```

19: if (isset($_POST['name'])) {
20:     $_SESSION['name'] = trim(mb_convert_kana($_POST['name'], "s"));
21: }
22: if (isset($_POST['kotoba'])) {
23:     $_SESSION['kotoba'] = trim(mb_convert_kana($_POST['kotoba'], "s"));
24: }

```

POST された値があれば、前後の空白を取り除いてセッション変数に代入します



入力値のチェック

次に改めてセッション変数に代入された値をチェックします。値は `empty()` でチェックし、値が空白や 0 ならば `$error[]` にエラーメッセージを追加します。値が入っていれば `$name` に代入します。`$name` の値は後で画面表示する際に使います。好きな言葉のチェックも同様に行います。(empty() 参照 P.423)

php

名前が入っているかどうかチェックする

«sample» session_form1/confirm.php

```

25: // 入力データの取り出しとチェック
26: $error = [];
27: // 名前
28: if (empty($_SESSION['name'])) { —— 名前が空でないかチェックします
29:     // 未設定のときエラー
30:     $error[] = "名前を入力してください。";
31: } else {
32:     // 名前を取り出す
33:     $name = $_SESSION['name']; —— セッション変数から名前を取り出します
34: }
```

名前が入っていない、好きな言葉が入っていないといった場合はエラーメッセージと「戻る」ボタンを表示します。「戻る」ボタンをクリックすると入力ページに戻りますが、入力されていた値は消えて最初の状態に戻っています。戻るボタンのタイプは "submit" ではなく "button" です。onclick 属性に移動先を指定しています。

php

エラーメッセージと「戻る」ボタンを表示する

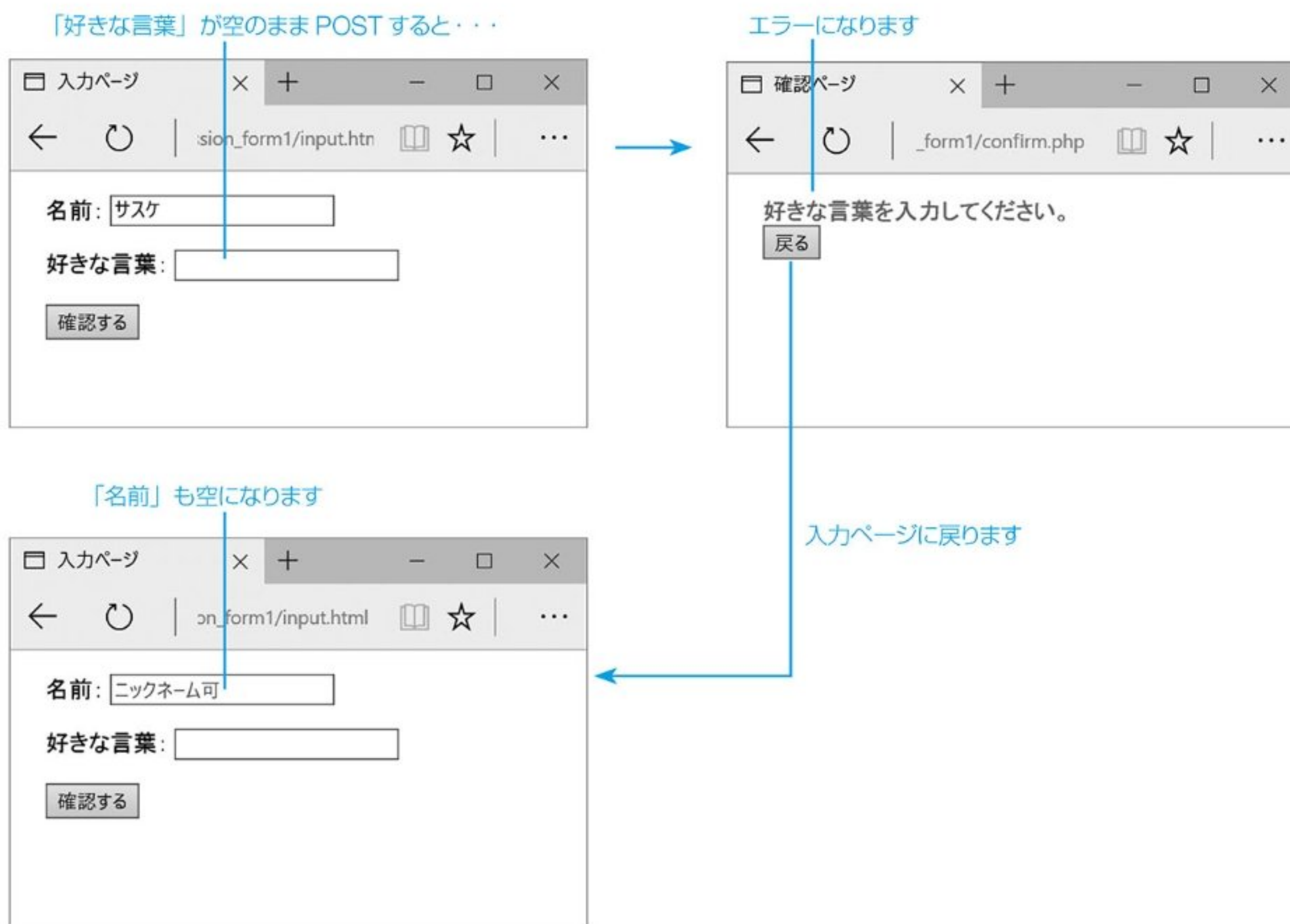
«sample» session_form1/confirm.php

```

55: <?php if (count($error)>0){ ?>
56:     <!-- エラーがあったとき -->
57:     <span class="error"><?php echo implode('<br>', $error); ?></span><br>
58:     <span>
59:         <input type="button" value="戻る" onclick="location.href='input.html'">
60:     </span>
61: <?php } else { ?>
```

エラーメッセージを
 タグで連結します

入力ページに戻ります



入力値と移動ボタンの表示

エラーがなかったとき名前と好きな言葉は \$name と \$kotoba に移してあります。これはユーザから POST された値なので、必ず HTML エスケープを行ってブラウザに表示します。HTML エスケープは最初に読み込んでおいた util.php の es() を使っています (P.307)。

「送信する」のボタンは POST メソッドは使わずに、「戻る」ボタンと同じように onclick に移動先を指定しています。

php 入力値と移動ボタンの表示

«sample» session_form1/confirm.php

```

61:      <?php } else { ?>
62:      <!-- エラーがなかったとき -->
63:      <span>
64:          名前:<?php echo es($name); ?><br>
65:          好きな言葉:<?php echo es($kotoba); ?><br>
66:          <input type="button" value=" 戻る " onclick="location.href='input.html'">
67:          <input type="button" value=" 送信する " onclick="location.href='thankyou.php'">
68:      </span>
69:      <?php } ?>

```

エラーがなかったときは、名前と言葉を HTML エスケープして表示します

クリックで完了ページへ移動します

NOTE

empty() が空と見なす値

empty() が空と見なすのは、""、0、0.0、"0"、NULL、FALSE、空の配列、値が設定されていない変数です。

完了ページを作る

完了ページでもセッションを利用するので、まずセッションを開始します。次にセッション変数から名前と言葉を取り出して、どちらも値が空でなければエラーなしという判断で表示しています。ここでも念のためにHTML エスケープを行ってからブラウザに表示します。最後にセッションを破棄するのも完了ページの大事な仕事です。

php 完了ページを作る

«sample» session_form1/thankyou.php

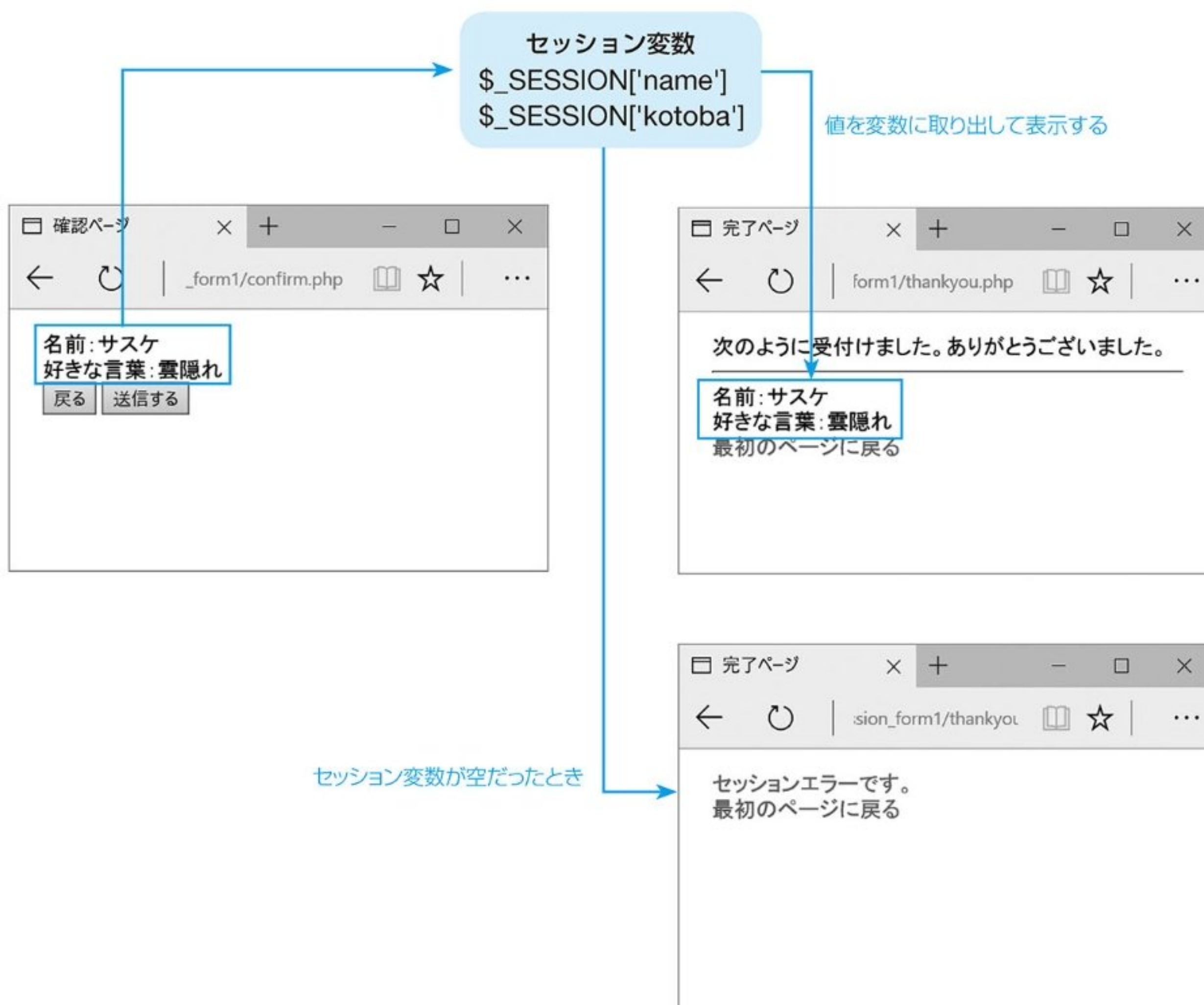
```
01: <?php
02: require_once("../lib/util.php");
03: // セッションの開始
04: session_start(); // セッションを開始します
05: $error = [];
06: // セッションのチェック
07: if (!empty($_SESSION['name']) && !empty($_SESSION['kotoba'])) {
08:     // セッション変数から値を取り出す
09:     $name = $_SESSION['name']; // セッション変数が空でないとき、値を変数に取り出します
10:     $kotoba = $_SESSION['kotoba'];
11: } else {
12:     $error[] = "セッションエラーです。";
13: }
14: // HTML を表示する前にセッションを破棄する
15: killSession();
16: ?>
17: // セッションを破棄します
18: <?php
19: // セッションを破棄する
20: function killSession(){
21:     // セッション変数の値を空にする
22:     $_SESSION = [];
23:     // セッションクッキーを破棄する
24:     if (isset($_COOKIE[session_name()])) {
25:         $params = session_get_cookie_params();
26:         setcookie(session_name(), '', time()-36000, $params['path']);
27:     }
28:     // セッションを破棄する
29:     session_destroy();
30: }
31: ?>
32:
33: <!DOCTYPE html>
34: <html lang="ja">
35: <head>
36:     <meta charset="utf-8">
37:     <title>完了ページ</title>
38:     <link href="../css/style.css" rel="stylesheet">
39: </head>
40: <body>
```

```

41: <div>
42:   <?php if (count($error)>0){ ?>
43:     <!-- エラーがあったとき -->
44:     <span class="error"><?php echo implode('<br>', $error); ?></span><br>
45:     <a href="input.html">最初のページに戻る </a>
46:   <?php } else { ?>
47:     <!-- エラーがなかったとき -->
48:     <span>
49:       次のように受け付けました。ありがとうございました。
50:     <hr>
51:     <span>
52:       名前:<?php echo es($name); ?><br>
53:       好きな言葉:<?php echo es($kotoba); ?><br>
54:       <a href="input.html">最初のページに戻る </a>
55:     </span>
56:   <?php } ?>
57: </div>
58: </body>
59: </html>

```

セッション変数から受け取っておいた値を表示します



HTML を表示する前にセッションを破棄する

完了ページでは「セッションを破棄する」という大事なことを行います。セッションは時間が経過すると自動的に破棄されますが、安全を確保するためにすぐに破棄することもできます。

セッションを破棄するには `session_destroy()` を実行します。ただ、`session_destroy()` だけでは完全ではなく、セッション変数 `$_SESSION` の値を空にし、セッションクッキーを削除する必要があります。セッションクッキーを削除するには、セッション名と保存パスで現在のクッキーを特定し、`setcookie()` を使って有効期限を現在より過去に設定します。過去の時刻は現在時刻は `time()` なので、`time()-36000` のようにして作ることができます。セッション名は `session_name()`、保存パスは `session_get_cookie_params()` から調べます。なお、クッキーについては次節で詳しく説明します (P.438)。

また、`session_start()` と同様に `session_destroy()` の前に空白や改行などを含めて HTML などがあるといけません。そこでセッション変数の値を取り出したならば、すぐに `session_destroy()` を実行します。

php セッションを破棄する

«sample» session_form1/thankyou.php

```
19: // セッションを破棄する
20: function killSession(){
21:     // セッション変数の値を空にする
22:     $_SESSION = [];
23:     // セッションクッキーを破棄する
24:     if (isset($_COOKIE[session_name()])){
25:         $params = session_get_cookie_params();
26:         setcookie(session_name(), '', time()-36000, $params['path']);
27:     }
28:     // セッションを破棄する
29:     session_destroy();
30: }
```

セッションクッキーの有効期限をマイナスにセットします



セキュリティ対策

セッション ID の置き換え (セッション固定攻撃などへの対応)

セッション固定攻撃などのセッションハイジャックの対処方法の1つとして、セッション ID の再発行があります。次のようにセッション開始後やログイン処理後に `session_regenerate_id()` を実行するとセッション ID が再発行されて置き換わります。セッション ID は変わりますが、セッション変数 `$_SESSION` の値は保たれています。

php セッション開始後にセッション ID を再発行する

```
01: <?php
02:     session_start();
03:     session_regenerate_id(true);
04: ?>
```

Section 10-3

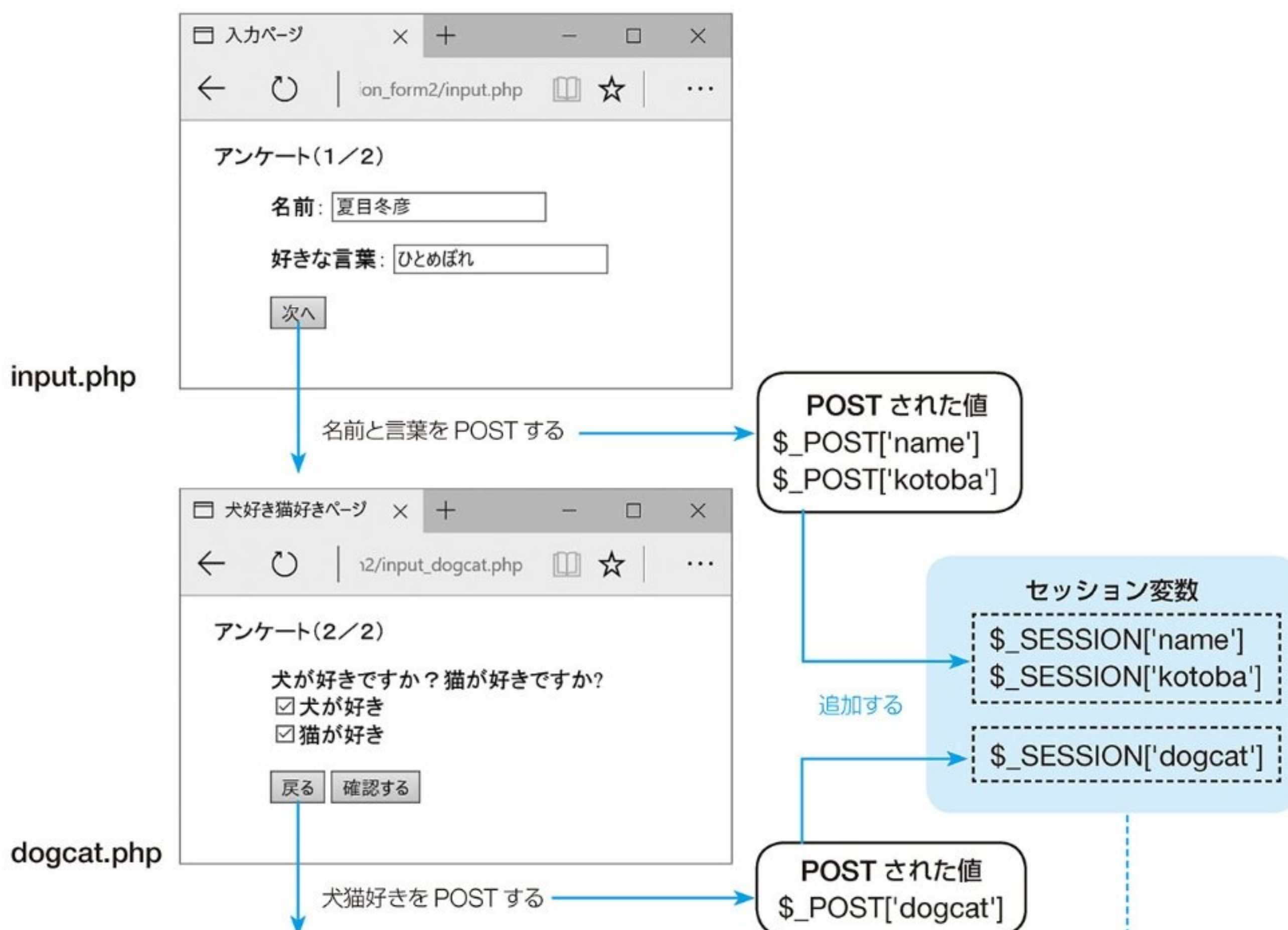
複数ページでセッション変数を利用する

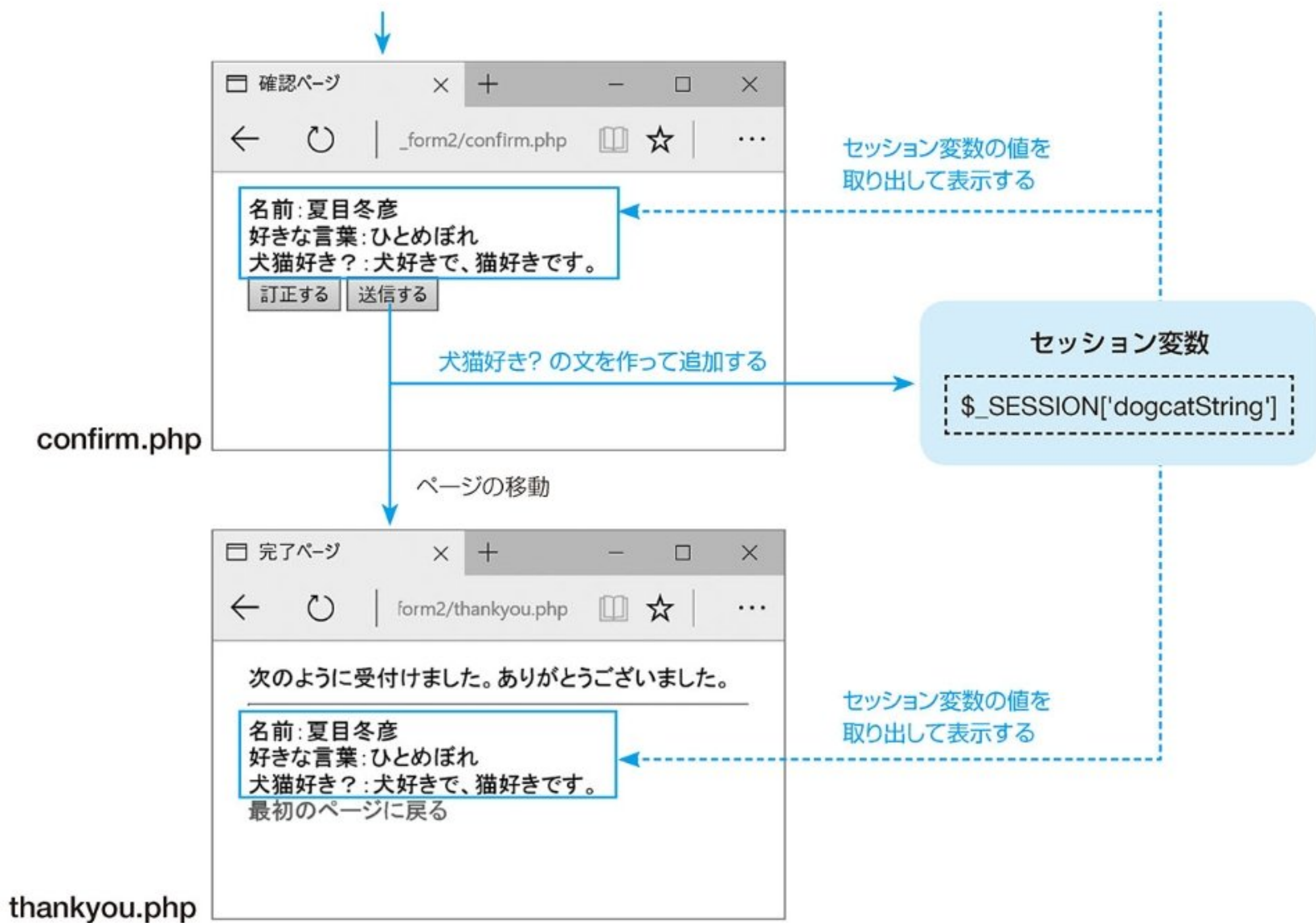
前節に続いてフォーム入力とセッションの組み合わせの例を取り上げます。前節のサンプルではフォーム入力が1ページだけでしたが、今回は入力が2ページに渡ります。入力の初期値をセッション変数で保ち、ページを戻って値を訂正できるようにします。

2ページに渡ってフォーム入力を行う

入力するページが増えても基本的な考え方は1ページの場合と同じです。フォーム入力の内容を `$_POST` 変数から取り出し、`$_SESSION` 変数に移していきます。`$_SESSION` 変数に現在の値が入っているので、この値を前のページに戻って修正できるようにします。最後の値の確認ページで「送信する」をクリックすると値が確定してセッションを終了し、セッション変数の値を破棄します。

なお、入力された値のチェックは毎ページごとに行うほうがよい場合がありますが、このサンプルでは最後にまとめて行っています。





名前と好きな言葉の入力ページ「アンケート（1 / 2）」を作る

前節のサンプルの入力ページは HTML コードだけだったので「input.html」の HTML ファイルでしたが、この節ではセッションを利用する PHP コードを追加します。したがって、拡張子を php に変更した「input.php」にします。

php 名前と好きな言葉を入力する入力ページを作る

«sample» session_form2/input.php

```

01: <?php
02: // セッションの開始
03: session_start(); // セッションを開始します
04: require_once("../lib/util.php");
05: // 確認ページから戻ってきたとき、セッション変数の値を取り出す
06: if (empty($_SESSION['name'])) {
07:     $name = "";
08: } else {
09:     $name = $_SESSION['name']; // セッション変数から値を受け取ります
10: }
11: if (empty($_SESSION['kotoba'])) {
12:     $kotoba = "";
13: } else {

```

```

14:     $kotoba = $_SESSION['kotoba']; —— セッション変数から値を受け取ります
15: }
16: ?>
17:
18: <!DOCTYPE html>
19: <html lang="ja">
20: <head>
21: <meta charset="utf-8">
22: <title>入力ページ</title>
23: <link href="../../../css/style.css" rel="stylesheet">
24: </head>
25: <body>
26: <div>                                     アンケートフォームを作ります
27:     アンケート (1 / 2) <br>
28:     <form method="POST" action="dogcat.php">
29:         <ul>
30:             <li><label>名前:
31:                 <input type="text" name="name" placeholder="ニックネーム可" value="<?php echo es($name) ?>";>
32:             </li></label></li>
33:             <li><label>好きな言葉:
34:                 <input type="text" name="kotoba" value="<?php echo es($kotoba) ?>";>
35:             </li></label></li>
36:             <li><input type="submit" value="次へ"></li>
37:         </ul>
38:     </form>
39: </div>
40: </body>
41: </html>

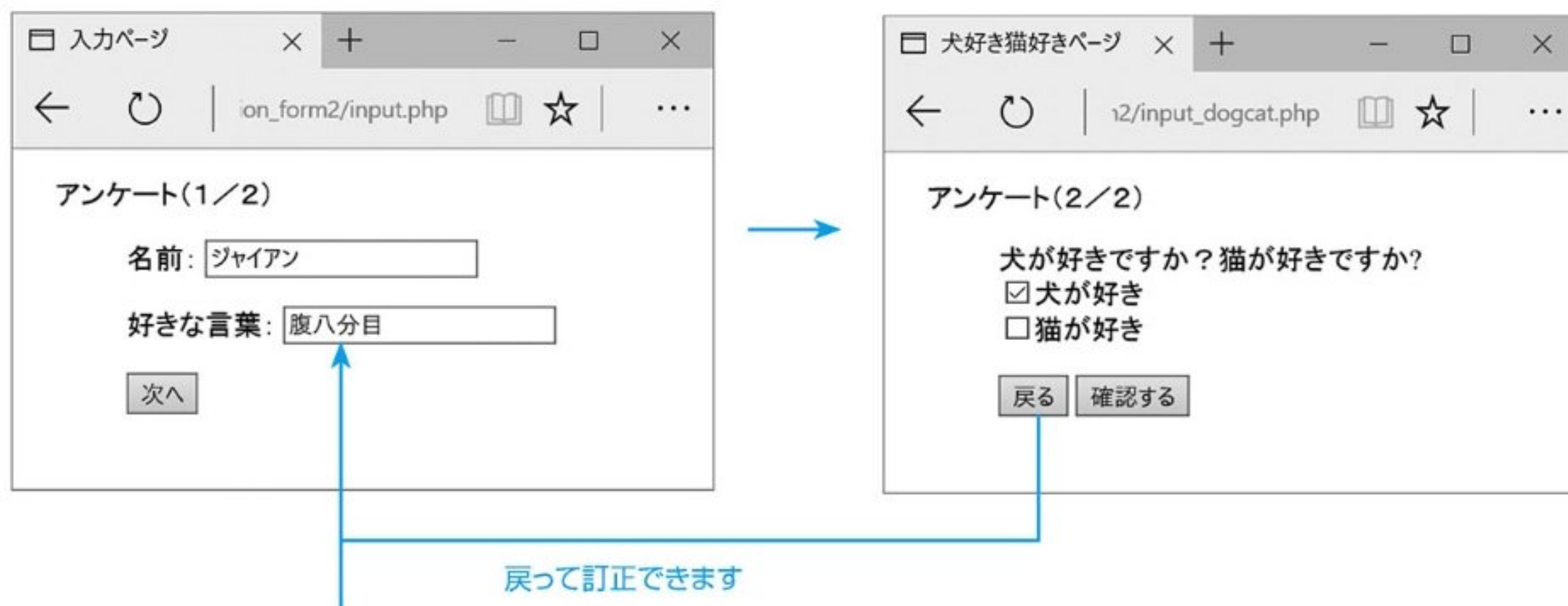
```

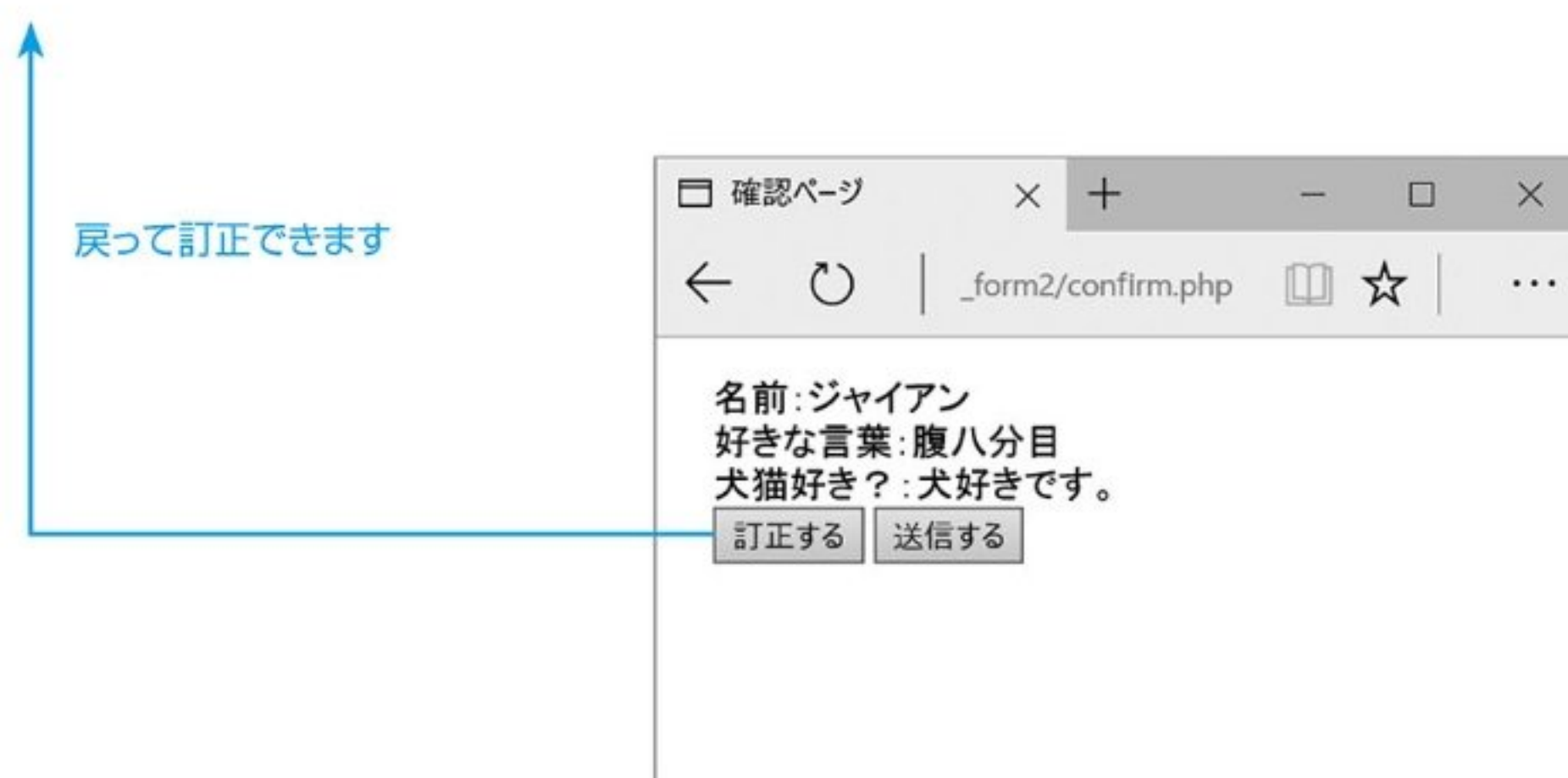
セッション変数から受け取った値を初期値にします

セッション変数の値を調べて入力フォームの初期値を決める

input.php の前半はセッション変数から値を取り出すコードです。このページを表示したときにセッション変数に値があれば、その値を「名前」、「好きな言葉」の初期値にします。

これはフォーム入力後にこのページに戻ってきた場合の対応です。具体的には「アンケート (2 / 2)」から「戻る」ボタンで戻ったときや確認ページから「訂正する」で戻ったときです。あるいは確認ページでエラーメッセージが出たときに最初のページに戻った場合もフィールドには現在の値が表示されています。





セッション変数を利用するので、まず最初に `session_start()` でセッションを開始します。セッション変数の `$_SESSION['name']`、`$_SESSION['kotoba']` の値が空かどうかを `empty()` で調べて、値があれば変数の `$name`、`$kotoba` にそれぞれの値を代入します。`empty()` が `true` のときは未設定の場合も含むので、`"` を代入して値を空にします。なお、`empty()` は値が `0` のときも `true` になります。ここでは、名前や好きな言葉が「0」という回答は認めていません (`empty()` P.423)。

php セッション変数に値があれば変数に入れる

«sample» session_form2/input.php

```
02: // セッションの開始
03: session_start();
04: require_once("../lib/util.php");
05: // 確認ページから戻ってきたとき、セッション変数から値を取り出す
06: if (empty($_SESSION['name'])) {
07:     $name = "";
08: } else {
09:     $name = $_SESSION['name'];
10: }
11: if (empty($_SESSION['kotoba'])) {
12:     $kotoba = "";
13: } else {
14:     $kotoba = $_SESSION['kotoba'];
15: }
```

セッション変数に値があれば変数に入れます

`$name`、`$kotoba` の値は `<input>` タグの `value` 属性に設定し、テキストフィールドの初期値として表示します。念のために表示する際には `es()` を使って HTML エスケープの処理を行っておきます。

php セッション変数から得た値をテキストフィールドの初期値にする

«sample» session_form2/input.php

```
30: <li><label> 名前:
31:     <input type="text" name="name" placeholder=" ニックネーム可 " value="<?php echo es($name) ?>">
32: </label></li>
33: <li><label> 好きな言葉:
34:     <input type="text" name="kotoba" value="<?php echo es($kotoba) ?>">
35: </label></li>
```

犬好き猫好きページ「アンケート（2／2）」を作る

このページは犬好き猫好きのアンケートをとるページですが、その前に「アンケート（1／2）」で入力された「名前」と「好きな言葉」を `$_POST` から値を取り出して `$_SESSION` に移すという大事な仕事があります。

「アンケート（1／2）」で入力された値の処理が終わったならば、犬好き猫好きのアンケートをとるチェックボックスの処理に移ります。

php チェックボックスで回答する犬好き猫好きページを作る

«sample» session_form2/dogcat.php

```

01: <?php
02: // セッションの開始
03: session_start();
04: require_once("../lib/util.php");
05:
06: // $_POST 変数に値があれば セッション変数に受け渡す
07: if (isset($_POST['name'])){
08:     $_SESSION['name'] = trim(mb_convert_kana($_POST['name'], "s"));
09: }
10: if (isset($_POST['kotoba'])){
11:     $_SESSION['kotoba'] = trim(mb_convert_kana($_POST['kotoba'], "s"));
12: }
13: // セッション変数に値があれば受け渡す
14: if (empty($_SESSION['dogcat'])){
15:     $dogcat = [];
16: } else {
17:     $dogcat = $_SESSION['dogcat'];
18: }
19: ?>
20:
21: <?php
22: // チェック状態にするかどうか決める
23: function checked(string $value, array $checkedValues) {
24:     // 選択する値に含まれているかどうか調べる
25:     $isChecked = in_array($value, $checkedValues);
26:     if ($isChecked) {
27:         // チェック状態にする
28:         echo "checked";
29:     }
30: }
31: ?>
32:
33: <!DOCTYPE html>
34: <html lang="ja">
35: <head>
36: <meta charset="utf-8">
37: <title>犬好き猫好きページ</title>
38: <link href="../css/style.css" rel="stylesheet">
39: </head>
40: <body>
41: <div>
42:     アンケート（2／2）<br>
43:     <form method="POST" action="confirm.php">
44:         <ul>
45:             <li><span>犬が好きですか？猫が好きですか？</span><br>
46:             <label><input type="checkbox" name="dogcat[]" value="犬"

```

「アンケート 1/2」から POST された値をセッション変数に入れます

前のページに戻って再度開いた場合にセッション変数に値が入っている可能性があります

アンケートフォームを作ります

```

47:         <?php checked("犬", $dogcat); ?> >犬が好き </label><br>
        <label><input type="checkbox" name="dogcat[]" value="猫"
        <?php checked("猫", $dogcat); ?> >猫が好き </label>
48:     </li>
49:     <input type="button" value=" 戻る " onclick="location.href='input.php' ">
50:     <input type="submit" value=" 確認する ">
51: </ul>
52: </form>
53: </div>
54: </body>
55: </html>

```

名前と好きな言葉を \$_POST から取り出して \$_SESSION に移す

前節の場合と同じように、POST された値を取り出してセッション変数に移します。セッションを利用するので、コードの最初でセッションを開始するのも忘れないでください。ここでは isset() で値が設定されているかどうかだけを確認し、空白の場合も含めて値をセッション変数に移します。

php 名前と好きな言葉を \$_POST から取り出して \$_SESSION に移す

«sample» session_form2/dogcat.php

```

06: // $_POST 変数に値があれば セッション変数に受け渡す
07: if (isset($_POST['name'])){
08:     $_SESSION['name'] = $_POST['name'];
09: }
10: if (isset($_POST['kotoba'])){
11:     $_SESSION['kotoba'] = $_POST['kotoba'];
12: }

```

犬好き猫好きの値をセッション変数から取り出す

すでに「アンケート (2/2)」を終わらせてこのページに戻ってきた場合に対応するために、\$_SESSION に犬好き猫好きの値があるかどうかを調べます。値があったならば、それをチェックボックスの初期値として設定します。

php 犬好き猫好きの値をセッション変数から取り出す

«sample» session_form2/dogcat.php

```

13: // セッション変数に値があれば受け渡す
14: if (empty($_SESSION['dogcat'])){
15:     $dogcat = [];
16: } else {
17:     $dogcat = $_SESSION['dogcat'];
18: }

```

チェックボックスのフォームを作る

アンケートフォームはチェックボックスで作ります。先にセッション変数から \$dogcat にチェックの現在の値を取り出しているので、チェックボックスのチェックの状態を設定します。

「戻る」ボタンはアンケート（1 / 2）のページに戻りますが、onclick で単に戻るだけでよく、現在の値を送って渡すといった必要はありません。「確認する」ボタンはチェックボックスで選んだ値を POST で送信します。選んだ値は配列 dogcat[] に入って POST されます。チェックボックスの作り方については、「Section9-2 チェックボックスを使う」を参照してください（P.364）。

php 犬好き猫好きを選ぶチェックボックスを作る

«sample» session_form2/dogcat.php

```
46: <label><input type="checkbox" name="dogcat[]" value="犬"
    <?php checked("犬", $dogcat); ?> >犬が好き </label><br>
47: <label><input type="checkbox" name="dogcat[]" value="猫"
    <?php checked("猫", $dogcat); ?> >猫が好き </label>
```

フォームから入力された値をチェックする確認ページを作る

最後にアンケート（1 / 2）、アンケート（2 / 2）の2ページで入力された値をチェックする「確認ページ」を作ります。このページでは、アンケート（1 / 2）で入力された値を \$_SESSION から取り出し、アンケート（2 / 2）で入力された値は \$_POST から取り出すことになります。

「確認ページ」はユーザに入力した値を確認してもらうという意味もありますが、プログラムでは値の妥当性をチェックします。値にエラーがあったならば該当のエラーメッセージと最初のページに「戻る」ボタンを表示します。

エラーがない場合は選ばれた値を表示し、このままでいいかどうかをユーザに決めてもらうために「訂正する」と「送信する」のボタンを並べて表示します。

確認ページ

← ↻ | _form2/confirm.php | ☆ | ...

名前: ジャイアン
好きな言葉: 腹八分目
犬猫好き?: 犬好きです。

訂正する 送信する

php 確認ページを作る

«sample» session_form2/confirm.php

```

01: <?php
02: // セッションの開始
03: session_start(); // セッションを開始します
04: require_once("../lib/util.php");
05: ?>
06:
07: <?php
08: // 文字エンコードの検証
09: if (!cken($_POST)){
10:     $encoding = mb_internal_encoding();
11:     $err = "Encoding Error! The expected encoding is " . $encoding ;
12:     $isError = true;
13:     // エラーメッセージを出して、以下のコードをすべてキャンセルする
14:     exit($err);
15: }
16: ?>
17:
18: <?php
19: // 入力データの取り出しとチェック
20: $error = [];
21: // セッション変数に値があれば受け渡す
22: if (empty($_SESSION['name'])){
23:     $error[] = "名前を入力してください。";
24: } else {
25:     $name = $_SESSION['name'];
26: }
27: if (empty($_SESSION['kotoba'])){
28:     $error[] = "好きな言葉を入力してください。";
29: } else {
30:     $kotoba = $_SESSION['kotoba'];
31: }
32:
33: // $_POST 変数に値があれば セッション変数に受け渡す
34: if (isset($_POST['dogcat'])){
35:     $dogcat = $_POST['dogcat'];
36:     // 値のチェック
37:     $diffValue = array_diff($dogcat, ["犬", "猫"]);
38:     // 規定外の値が含まれていなければ OK
39:     if (count($diffValue)>0){
40:         $error[] = "犬好き猫好きの回答にエラーがありました。";
41:         $_SESSION['dogcat'] = [];
42:     } else {
43:         $dogcatString = implode("好きで、", $dogcat) . "好きです。";
44:         $_SESSION['dogcatString'] = $dogcatString;
45:         $_SESSION['dogcat'] = $dogcat;
46:     }
47: } else {
48:     $dogcatString = "どちらも好きではありません。";
49:     $_SESSION['dogcatString'] = $dogcatString;
50:     $_SESSION['dogcat'] = [];
51: }
52: ?>
53:
54: <!DOCTYPE html>
55: <html lang="ja">
56: <head>
57:     <meta charset="utf-8">

```

「アンケート 1/2」での入力

「アンケート 2/2」での入力

セッション変数に新規に
追加します

Part 3

Chapter

8

Chapter

9

Chapter

10

Chapter

11

```

58:    <title>確認ページ</title>
59:    <link href="../../css/style.css" rel="stylesheet">
60: </head>
61: <body>
62: <div>
63: <form>                                $Error が空かどうかで分岐します
64: <?php if (count($error)>0){ ?>
65:     <!-- エラーがあったとき -->          エラーを改行の <br> タグで区切って表示します
66:     <span class="error"><?php echo implode('<br>', $error); ?></span><br>
67:     <span>
68:         <input type="button" value=" 戻る " onclick="location.href='input.php'">
69:     </span>
70: <?php } else { ?>
71:     <!-- エラーがなかったとき -->
72:     <span>
73:         名前:<?php echo es($name); ?><br>
74:         好きな言葉:<?php echo es($kotoba); ?><br>
75:         犬猫好き?:<?php echo es($dogcatString); ?><br>
76:         <input type="button" value=" 訂正する " onclick="location.href='input.php'">
77:         <input type="button" value=" 送信する " onclick="location.href='thankyou.php'">
78:     </span>
79: <?php } ?>
80: </form>
81: </div>
82: </body>
83: </html>

```

セッション変数から値を取り出してチェックする

一番最初に入力してもらった名前と好きな言葉は `$_SESSION` に入れてありますが、値が入っているかどうかをチェックしていないので `empty()` でチェックします。値が入っていれば `$name`、`$kotoba` に取り出し、値が入ってなければエラーメッセージを配列 `$error` に追加します。

php 名前と好きな言葉をセッション変数から取り出してチェックする

«sample» session_form2/confirm.php

```

21: // セッション変数に値があれば受け渡す
22: if (empty($_SESSION['name'])) {
23:     $error[] = "名前を入力してください。";
24: } else {
25:     $name = $_SESSION['name'];
26: }
27: if (empty($_SESSION['kotoba'])) {
28:     $error[] = "好きな言葉を入力してください。";
29: } else {
30:     $kotoba = $_SESSION['kotoba'];
31: }

```

犬好きか猫好きかの値をセッション変数に移す

このページには犬好きか猫好きかのチェックボックスの値が POST されて渡されているので、`$_POST` の値を取り出してチェックします。`$_POST` に配列の値が入っていれば `$_SESSION` に値を移します。配列が入っ

ていた場合も規定外の値が含まれていないかどうかを `array_diff()` を使ってチェックします (P.233)。

値にエラーがなければ `implode()` を利用して配列 `$dogcat` の要素を「犬好きで、猫好きです。」のように連結した文にして `$dogcatString` に納めます。もし、`$dogcat` に「犬」、「猫」のどちらか1個しか入っていない場合、`implode()` は値をそのまま戻すので「犬好きです。」あるいは「猫好きです。」の文になります。そして、配列が空の場合は「どちらも好きではありません。」が `$dogcatString` に入ります。作成した文はセッション変数 `$_SESSION['dogcatString']` に保存します。

php 犬好き猫好きのチェックボックスの値を取り出してチェックする

«sample» session_form2/confirm.php

```

33: // $_POST 変数に値があれば セッション変数に受け渡す
34: if (isset($_POST['dogcat'])){
35:     $dogcat = $_POST['dogcat'];
36:     // 値のチェック
37:     $diffValue = array_diff($dogcat, ["犬", "猫"]);
38:     // 規定外の値が含まれていなければ OK
39:     if (count($diffValue)>0){
40:         $error[] = "犬好き猫好きの回答にエラーがありました。";
41:         $_SESSION['dogcat'] = [];
42:     } else {
43:         $dogcatString = implode("好きで、", $dogcat) . "好きです。";
44:         $_SESSION['dogcatString'] = $dogcatString;
45:         $_SESSION['dogcat'] = $dogcat;
46:     }
47: } else {
48:     $dogcatString = "どちらも好きではありません。";
49:     $_SESSION['dogcatString'] = $dogcatString;
50:     $_SESSION['dogcat'] = [];
51: }

```

——— 作った文をセッション変数に保存します
 ——— POST された値をセッション変数に保存します

確認ページ

名前:ももこ
 好きな言葉:桃栗三年柿八年
 犬猫好き?:犬好きで、猫好きです。

チェックボックスを
両方チェックしているとき

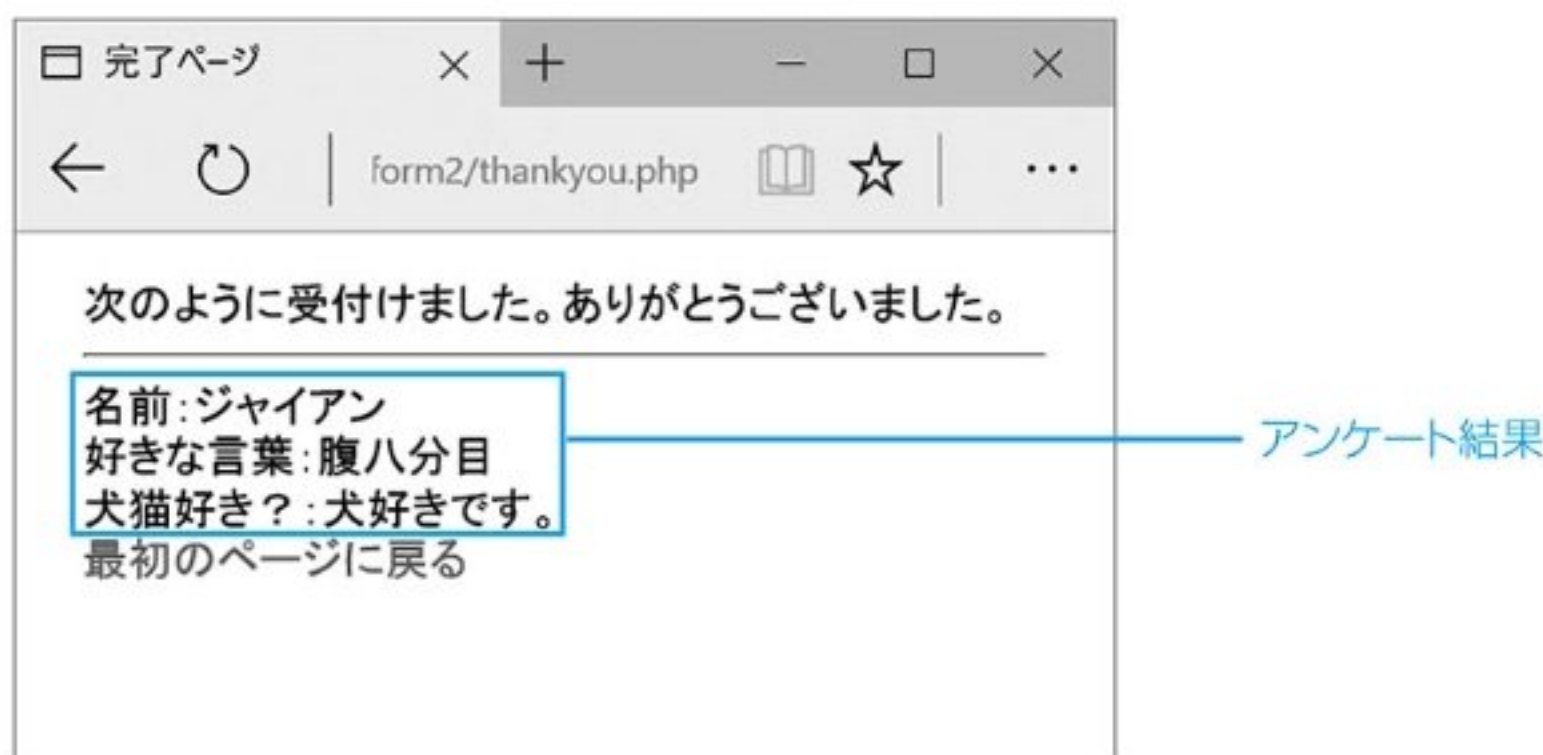
確認ページ

名前:ももこ
 好きな言葉:桃栗三年柿八年
 犬猫好き?:どちらも好きではありません。

チェックボックスを
どちらもチェックしていないとき

完了ページを作る

最後に入力結果を表示してセッションを終了する完了ページを作ります。アンケート結果は `$_SESSION` に入っているのを値を変数に取り出します。セッション変数から値を取り出したならばセッションを破棄し、変数に取り出した値をブラウザに表示します。セッションを破棄したので、「最初のページに戻る」で戻るとアンケートの入力フォームは空から始まります。



php すべての値をセッション変数から取り出してセッションを終了する

«sample» session_form2/thankyou.php

```
01: <?php
02: require_once("../lib/util.php");
03: // セッションの開始
04: session_start(); // セッションを開始します
05: // セッションのチェック
06: $error = [];
07: if (!empty($_SESSION['name']) && !empty($_SESSION['kotoba'])) {
08:     // セッション変数から値を取り出す
09:     $name = $_SESSION['name'];
10:     $kotoba = $_SESSION['kotoba'];
11:     $dogcatString = $_SESSION['dogcatString'];
12: } else {
13:     // セッション変数が空だったとき
14:     $error[] = "セッションエラーです。";
15: }
16: // HTML を表示する前にセッションを終了する
17: killSession()
18: ?>
19:
20: <?php
21: // セッションを破棄する
22: function killSession(){
23:     // セッション変数の値を空にする
24:     $_SESSION = [];
25:     // セッションクッキーを破棄する
26:     if (isset($_COOKIE[session_name()])) {
27:         $params = session_get_cookie_params();
28:         setcookie(session_name(), '', time()-36000, $params['path']);
29:     }
30:     // セッションを破棄する
```

```

31:     session_destroy();
32: }
33: ?>
34:
35: <!DOCTYPE html>
36: <html lang="ja">
37: <head>
38:     <meta charset="utf-8">
39:     <title>完了ページ</title>
40:     <link href="../../css/style.css" rel="stylesheet">
41: </head>
42: <body>
43: <div>
44:     <?php if (count($error)>0){ ?>
45:         <!-- エラーがあったとき -->
46:         <span class="error"><?php echo implode('<br>', $error); ?></span><br>
47:         <span>
48:             <input type="button" value="最初のページに戻る" onclick="location.href='input.php'">
49:         </span>
50:     <?php } else { ?>
51:         <!-- エラーがなかったとき -->
52:         次のように受けました。ありがとうございました。
53:         <hr>
54:         <span>
55:             名前:<?php echo es($name); ?><br>
56:             好きな言葉:<?php echo es($kotoba); ?><br>
57:             犬猫好き?:<?php echo es($dogcatString); ?><br>
58:             <a href="input.php">最初のページに戻る</a>
59:         </span>
60:     <?php } ?>
61: </div>
62: </body>
63: </html>

```

エラーを改行の
 タグで区切って表示します

アンケート結果を表示します



セキュリティ対策

トークンを利用して遷移チェックする (CSRF 対策)

セッション変数と POST 変数を利用して、正しい遷移で Web ページが開いていたかどうかを確認する方法があります。具体的にはページを遷移する前に乱数でワンタイムトークンを生成してセッション変数に保管し、遷移先にも POST します。遷移先ではセッション変数のトークンと POST 変数に入っているトークンを比較して一致すれば正しい遷移が行われたと判断します。トークンは次のコードで生成できます。

php ワンタイムトークンを生成する

```

01: // トークン (乱数) を生成
02: $bytes = openssl_random_pseudo_bytes(16);
03: // 16 進数に変換
04: $token = bin2hex($bytes);

```

Section 10-4

クッキーを使う

クッキーとセッションはよく似た機能ですが、セッションは複数ページに渡って利用する変数を保持するという目的で利用されるのに対し、クッキーはブラウザから離れてもユーザの値を保管しておく目的で利用されます。この節では、クッキーで値を保存し、その値を取り出す簡単な例を示します。

クッキーに保存する

クッキーはセッションと違って、利用開始のメソッドを実行する必要がありません。setcookie() を使ってすぐに利用できます。ただし、セッションと同じように setcookie() を実行するよりも前に空白、空行、HTML コードなどの出力があるとエラーになります。また、ユーザによってはブラウザでのクッキーの利用を許可していないことがあるので、その点にも注意が必要です。

ではさっそくクッキーに値を保存し、確認する流れを簡単な例を見てみましょう。次の例では開いたページ (set_cookie.php) で \$message に入っているメッセージを保存します。そして、移動先のページ (check_cookie.php) でクッキーに保存した値を取り出して確認します。クッキーは配列 \$_COOKIE で値をやり取りします。



php クッキーに値を保存する

«sample» cookie_values/set_cookie.php

```

01: <?php
02: // クッキーに保存する値を準備する
03: $message = "ハロー";
04: // クッキーに値を代入する（ブラウザを閉じるまで有効）
05: $result = setcookie("message", $message);
06: ?>
07:
08: <!DOCTYPE html>
09: <html lang="ja">
10: <head>
11: <meta charset="utf-8">
12: <title>クッキー保存ページ</title>
13: <link href="../../css/style.css" rel="stylesheet">
14: </head>
15: <body>
16: <div>
17:     <?php
18:     if ($result){
19:         echo "クッキーを保存しました。", "<hr>";
20:         echo '<a href="check_cookie.php">クッキーを確認するページへ</a>';
21:     } else {
22:         echo '<span class="error">クッキーの保存でエラーがありました。</span>', "<br>";
23:     }
24:     ?>
25: </div>
26: </body>
27: </html>

```

クッキーに保存します。
これより前に空白行や HTML コードがあってははいけません。

クッキーへの保存が成功したとき true になります

setcookie() の書式

先にも書いたようにクッキーに値を保存する使用するメソッドは `setcookie()` です。`setcookie()` の書式は次のとおりです。`$result` には、クッキーの保存が成功したら `true`、失敗したら `false` が戻ります。

書式 クッキーを保存する

```
$result = setcookie ( クッキー名 , 値 , 期限 , パス , ドメイン , セキュア , HTTP オンリー );
```

第1引数にクッキーの名前を指定し、第2引数に保存する値を指定します。クッキー名は、クッキーから値を取り出したり削除したりする際の配列のキーであり、クッキー自体を特定するための名前です。すでに同じ名前のクッキーが存在していたならば値を上書きして更新することになります。

第3引数はクッキーの有効期限です。1970年1月1日午前0時（GMT）から起算した経過秒数（Unix タイムスタンプ）で指定します。`time()` が現在の起算秒数を返すので、`time()+60` ならば作成後1分が有効期限です。有効期限を3日にしたければ、`time()*60*60*24*3` のように指定します。有効期限を省略するか、0を指定するとブラウザを閉じるまでが有効期限になります。

「パス、ドメイン、セキュア、HTTP オンリー」は通常は指定しなくても構いません。パス、ドメインはクッ

キーが有効な範囲を示します。パス、ドメインを省略すると現在のページが存在するディレクトリ、サブドメインの範囲でのみ有効です。セキュアを true にすると https 経由でのみクッキーを送り返すようになります。デフォルトは false です。HTTP オンリーを true にすると JavaScript からのアクセスを禁止します。このほうが安全ですが、未対応のブラウザがあるため初期値は false になっています。

クッキーに値を保存する

この例ではクッキーに "message" の名前で \$message の値を保存しています。有効期限を省略しているので、ブラウザを閉じるとクッキーは削除されます。

php クッキーに "message" の名前で " ハロー " と保存する

«sample» cookie_values/set_cookie.php

```
02: // クッキーに保存する値を準備する
03: $message = " ハロー ";
04: // クッキーに値を代入する (ブラウザを閉じるまで有効)
05: $result = setcookie("message", $message); ————— 期限を省略しているのでブラウザを閉じるまで有効です
```

クッキーから値を取り出す

クッキーが有効になるのはページをロードした時点です。最初のページの「クッキーを確認するページへ」をクリックして、いったんページを移動するとグローバル変数 \$_COOKIE からクッキーの値を取り出せるようになります。保存したときに "message" という名前を付けたので、\$_COOKIE["message"] で値を取り出します。

これまでの \$_POST や \$_SESSION から値を取り出したときと同じように、isset() で値が設定されているかどうかを確認し、\$_COOKIE["message"] の中身を変数 \$message に取り出します。クッキーの値をブラウザに表示する際には、必ず HTML エスケープを通します。ここではこれまでと同じように util.php で定義しておいた es() を使って、es(\$message) のようにしています。(P.307)

php クッキーから値を取り出す

«sample» cookie_values/check_cookie.php

```
01: <?php
02: require_once("../lib/util.php");
03: ?>
04:
05: <!DOCTYPE html>
06: <html lang="ja">
07: <head>
08:   <meta charset="utf-8">
09:   <title>クッキー確認ページ</title>
10:   <link href="../css/style.css" rel="stylesheet">
11: </head>
12: <body>
13: <div>
14:   <?php
15:   // クッキー変数を調べる
```

```

16: echo "クッキーを確認しました。", "<br>";
17: if(isset($_COOKIE["message"])){
18:     // クッキーの値を取り出す
19:     $message = $_COOKIE["message"]; —— クッキーから値を取り出します
20:     echo "クッキーの値：", es($message), "<hr>"; —— HTML エスケープを行ってからブラウザに表示します
21:     echo '<a href="delete_cookie.php">クッキーを削除する </a>';
22: } else {
23:     echo "クッキーはありません。", "<hr>";
24:     echo '<a href="set_cookie.php">クッキーを設定するページへ </a>';
25: }
26: ?>
27: </div>
28: </body>
29: </html>

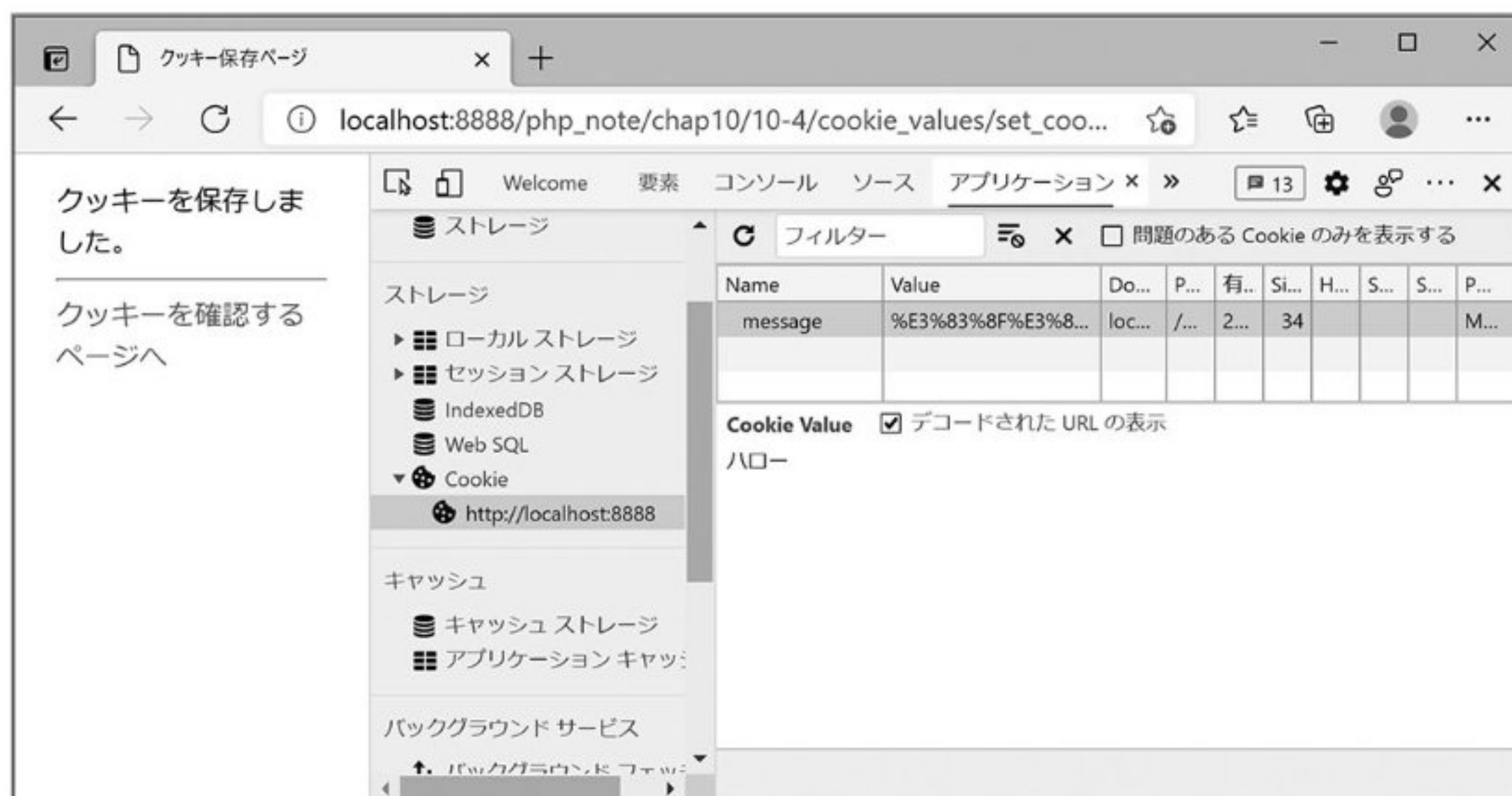
```



セキュリティ対策

クッキーは簡単に見ることができ、改ざんもできる

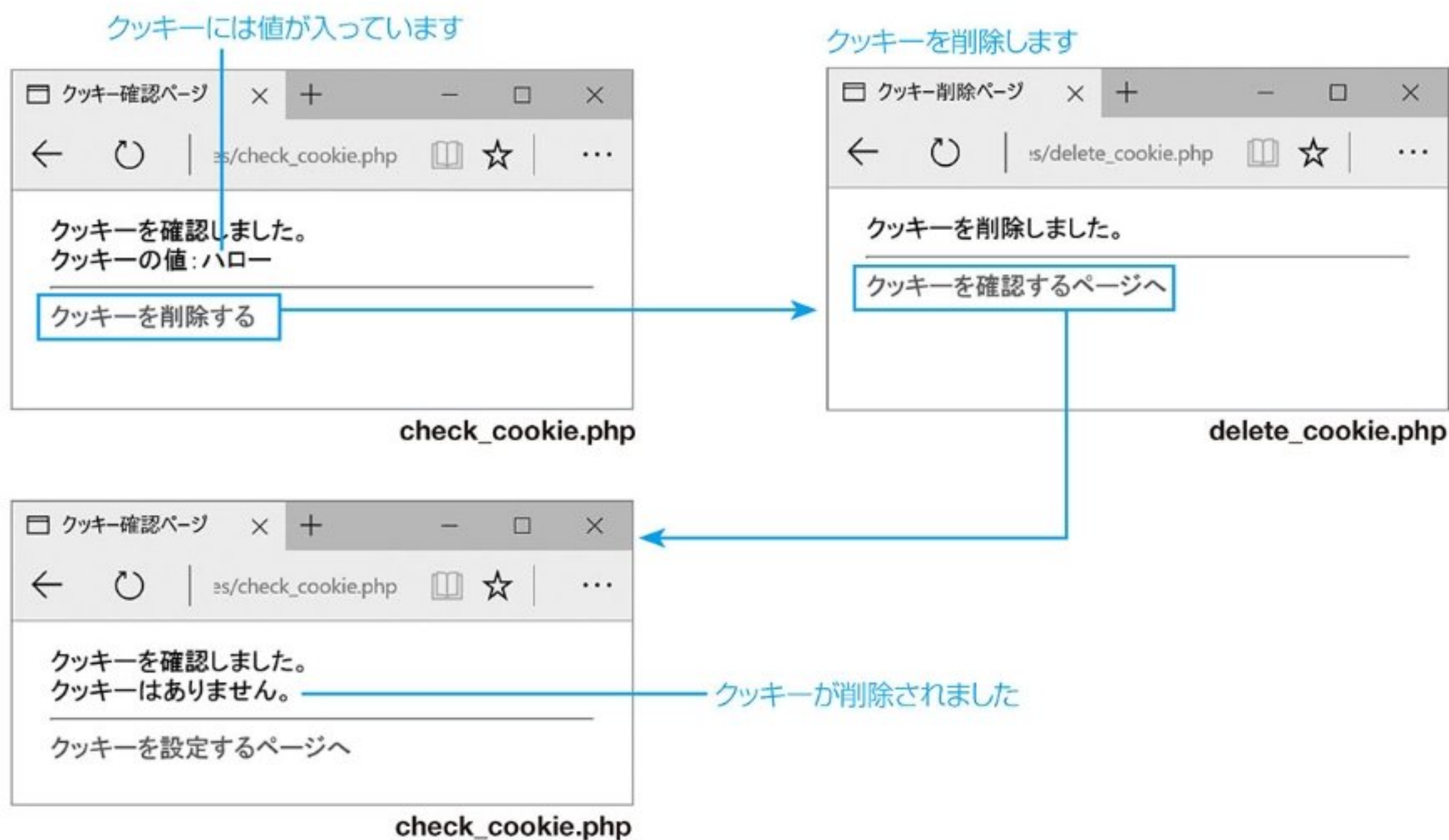
クッキーはユーザのコンピュータに保存され、その中身はブラウザで簡単に見ることができるので重要な値を保存してはいけません。また、ブラウザに表示する場合は HTML エスケープを行ってください。



クッキーの有効期限とクッキーの削除

書式の説明でも書いたように、クッキーの有効期限はクッキーを保存する `setcookie()` の第3引数で指定します。クッキーを削除するには、削除したいクッキーの有効期限を過去の期日で更新します。過去の期日とは、`time()-3600` といった値です。

前節ではセッションを破棄する際に同時にセッションクッキーを削除しています。そこでは、クッキー名に加えて保存パスを指定して削除するクッキーを特定しています (P.425)。



php クッキーを削除する

«sample» cookie_values/delete_cookie.php

```

01: <?php
02: // クッキーを削除する
03: $result = setcookie("message", "", time()-3600);
04: ?>
05:
06: <!DOCTYPE html>
07: <html lang="ja">
08: <head>
09: <meta charset="utf-8">
10: <title>クッキー削除ページ</title>
11: <link href="../../css/style.css" rel="stylesheet">
12: </head>
13: <body>
14: <div>
15:     <?php
16:     if ($result){
17:         echo "クッキーを削除しました。", "<hr>";
18:         echo '<a href="check_cookie.php">クッキーを確認するページへ</a>';
19:     } else {
20:         echo '<span class="error">クッキーの削除でエラーがありました。</span>', "<br>";
21:     }
22:     ?>
23: </div>
24: </body>
25: </html>

```

有効期限を過去に変更することで、クッキーを削除します

NOTE

試しに作るクッキーは有効期限を短くするか省略しておく

有効期限を省略するとブラウザを閉じればクッキーが削除されます。有効期限を省略しておくと試しに作ったクッキーが残ってしまう面倒がありません。

Section 10-5

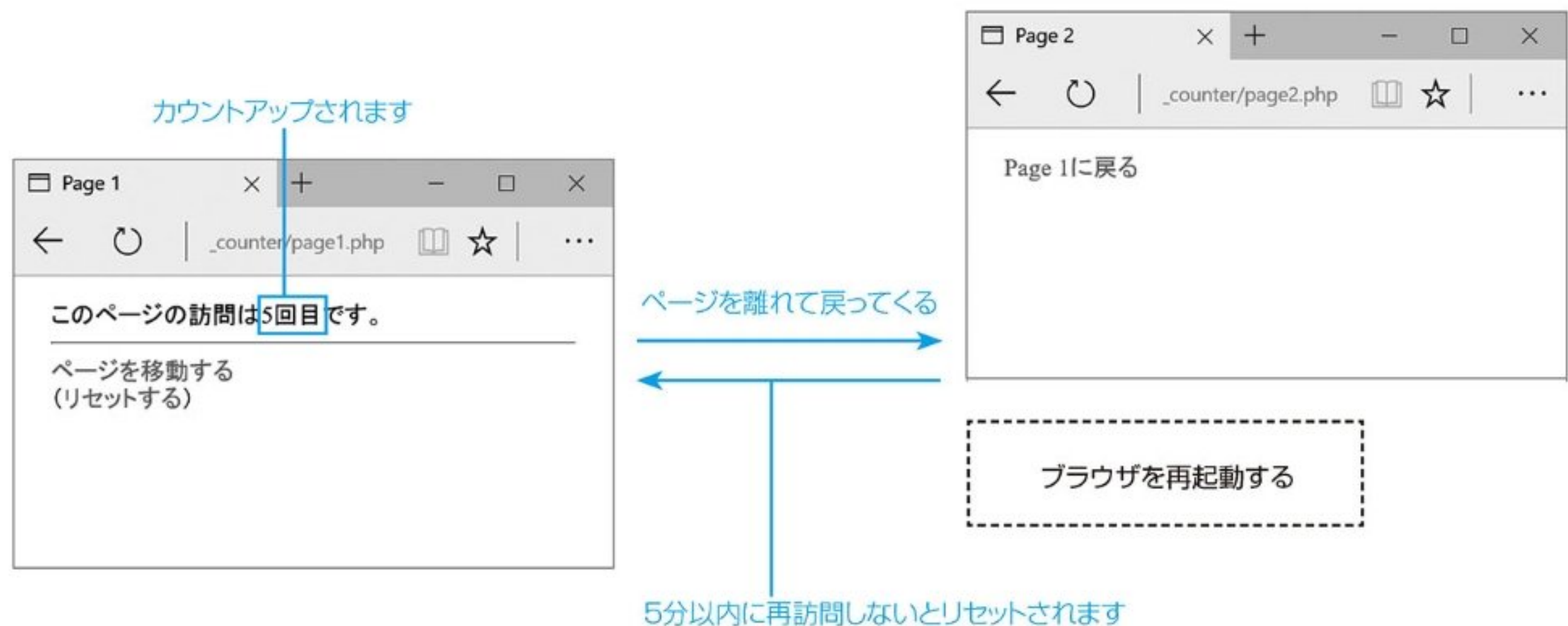
クッキーで訪問カウンタを作る

クッキーを使った例としてページを訪れた回数をカウントアップするカウンタを作ってみましょう。1つ目の例は訪問回数のカウントアップ、2つ目の例は訪問日時も保存します。2つ目の例では配列をクッキーに保存する方法を取り上げます。

訪問カウンタを作る

クッキーを利用して訪問カウンタを作ります。クッキーに保存した値を直接書き替えることはできないので、ページを開くたびにクッキーに保存してある値をいったん取り出し、値を更新して同名のクッキーで上書き保存します。

通常、訪問カウンタの有効期限は長期間に設定しますが、ここではテスト用として5分間に設定しています。つまり、5分以内に再訪問しないとクッキーが破棄され、カウンタがリセットされることになります。なお、ブラウザの戻るボタンでページを戻った場合はページがリロードしないのでカウントアップされません。



php 訪問カウンタを作る

«sample» visited_counter/page1.php

```

01: <?php
02: require_once("../lib/util.php");
03: // クッキーの値を取り出す
04: if (isset($_COOKIE["visitedCount"])){
05:     // 現在のカウンタの値を取り出す
06:     $visitedCount = $_COOKIE["visitedCount"]; ——— クッキーから前回の訪問数を取り出します
07: } else {
08:     // クッキーがないのでカウンタに初期値を設定する
09:     $visitedCount = 0;
10: }
11: // クッキーの値をカウントアップする (テスト用に 5 分間有効)
12: $result = setcookie("visitedCount", ++$visitedCount, time()+60*5);
13: ?>
14:                                     訪問数をカウントアップして、クッキーに保存し直します
15: <!DOCTYPE html>
16: <html lang="ja">
17: <head>
18: <meta charset="utf-8">
19: <title>Page 1</title>
20: <link href="../../css/style.css" rel="stylesheet">
21: </head>
22: <body>
23: <div>
24:     <?php
25:     if ($result) {
26:         echo "このページの訪問は ", es($visitedCount), " 回目です。", "<hr>";
27:         echo '<a href="page2.php"> ページを移動する </a>', "<br>";
28:         echo ' (<a href="reset_counter.php"> リセットする </a>) ';
29:     } else {
30:         echo '<span class="error"> クッキーが利用できませんでした。</span>';
31:     }
32:     ?>
33: </div>
34: </body>
35: </html>

```

1 を加算した訪問数を表示します

カウンタの値を取り出す

すでに説明したようにカウンタをカウントアップするには、クッキーに保存してある値をいったん取り出します。クッキーには有効期限があるので、クッキーの値を調べる場合は値が存在するかどうかを確認します。

ここではカウンタを "visitedCount" の名前のクッキーで保存しているので、`$_COOKIE["visitedCount"]` が設定済みかどうかをチェックし、設定済みならば値を `$visitedCount` に取り出し、値がなければ `$visitedCount` を 0 に設定します。

php カウンタの値を取り出す

«sample» visited_counter/page1.php

```

03: // クッキーの値を取り出す
04: if (isset($_COOKIE["visitedCount"])){ —— 訪問数のクッキーが存在するかどうかチェックします
05:     // 現在のカウンタの値を取り出す
06:     $visitedCount = $_COOKIE["visitedCount"];
07: } else { —— 前回までの訪問数を取り出します
08:     // クッキーがないのでカウンタに初期値を設定する
09:     $visitedCount = 0;
10: }
```

カウンタをカウントアップして保存する

カウンタのカウントアップは、クッキーを保存する際に `++$visitedCount` を実行して同時にやってしまいます（++ ➦ P.72）。カウンタの初期値は 0 ですが、最初の訪問が 1 回目として記録されます。一定期間訪問しなかったならばカウンタをリセットするテストを行うために、クッキーの有効時間を短く 5 分（`time()+60*5`）に設定しています。5 分以内に再訪問しないとクッキーが破棄されてカウンタがリセットされます。

php クッキーの値をカウントアップする（テスト用に 5 分間有効）

«sample» visited_counter/page1.php

```

12: $result = setcookie("visitedCount", ++$visitedCount, time()+60*5);
```

移動先のページ

移動先のページにはカウンタが設定してある Page 1（page1.php）に戻るリンクがあるだけで、PHP コードはありません。いったん Page1 から移動することで、クッキーへの保存が有効になります。

php 移動先のページ

«sample» visited_counter/page2.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04:     <meta charset="utf-8">
05:     <title>Page 2</title>
06:     <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:     <a href="page1.php">Page 1 に戻る </a>
11: </div>
12: </body>
13: </html>
```

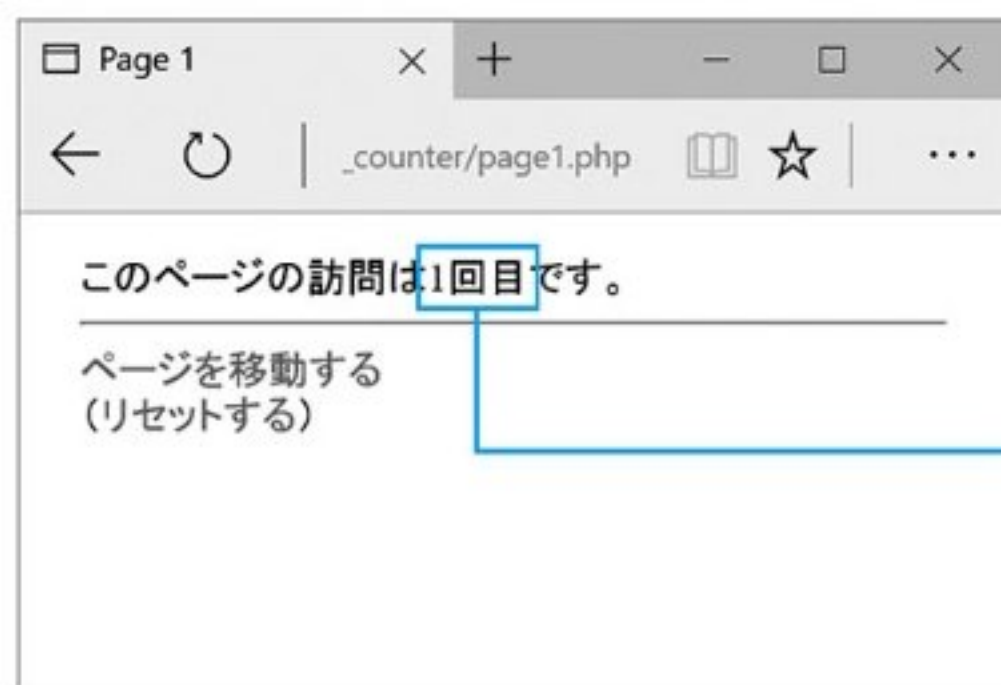
カウンタの値をリセットする

カウンタの値をリセットするには、クッキーを削除してしまってもかまいませんが（➦ P.441）、ここではカウンタの値を 0 にしてクッキーを保存しています。

リセットページへ移動します



訪問数を 0 にリセットします



1ページ目に戻る

1回目の訪問

php カウンタの値を 0 にリセットする

«sample» visited_counter/reset_counter.php

```

01:  <?php
02:  // カウントをリセットする (テスト用に 5 分間有効)
03:  $result = setcookie("visitedCount", 0, time()+60*5);
04:  ?>
05:                                     カウンタを 0 にしてクッキーを上書き保存します
06:  <!DOCTYPE html>
07:  <html lang="ja">
08:  <head>
09:  <meta charset="utf-8">
10:  <title> リセットページ </title>
11:  <link href="../../../css/style.css" rel="stylesheet">
12:  </head>
13:  <body>
14:  <div>
15:  <?php
16:  if ($result){
17:  echo " カウンタをリセットしました。", "<hr>";
18:  echo '<a href="page1.php">Page 1 に戻る </a>';
19:  } else {
20:  echo '<span class="error"> カウンタのリセットでエラーがありました。 </span>';
21:  }
22:  ?>
23:  </div>
24:  </body>
25:  </html>

```

クッキーに配列を保存する

次の例では訪問回数だけでなく、訪問日時も保存します。2個の値をそれぞれ個別にクッキーに保存すればよいのですが、ここでは2個の値を配列として保存する方法を紹介します。

配列を保存する書式

クッキーには `setcookie(クッキー名, 配列)` のように配列を直接保存することはできません。クッキーに配列を保存する場合は、`クッキー名[キー]` の書式を使って要素ごとに値を保存します。この書式では、クッキー名 `"キー"` ではなく、`"クッキー名[キー]"` のように第1引数全体を1つの文字列にする点に注意してください。

書式 クッキーに配列の値を保存する

```
$result = setcookie("クッキー名[キー]", 値);
```

一方、`$_COOKIE` から値を取り出す場合は保存したときのように `$_COOKIE("クッキー名[キー]")` のように指定せず、`$_COOKIE(クッキー名)` でいったん配列を取り出してから各要素にアクセスします。

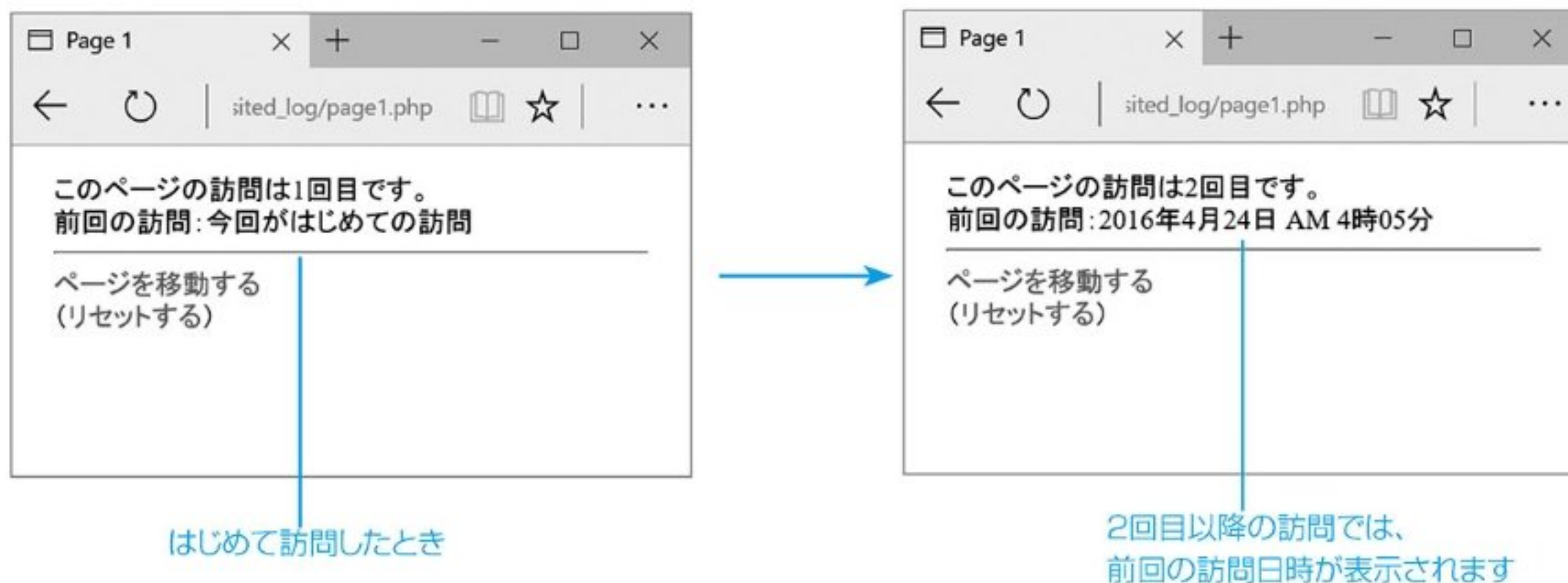
NOTE

配列をストリングに変換して保存する

クッキーを配列の書式で保存しても、実際には値ごとに個別のクッキーに保存されます。保存できるクッキーの個数はブラウザによって制限があるので、配列を文字列に変換して1個のクッキーで済ませる方法を次節で説明します。

訪問回数と日時を配列で保存する

回数 (`$counter`) と日時 (`$time`) の2つの値があるので、これを `[$counter, $time]` の配列にしてクッキー `"visitedLog"` に保存します。



php クッキーに訪問回数と日時を保存する

«sample» visited_log/page1.php

```

01: <?php
02: require_once("../lib/util.php");
03: // クッキーの値を取り出す
04: if (isset($_COOKIE["visitedLog"])){
05:     // 訪問ログの値を取り出す
06:     $logdata = $_COOKIE["visitedLog"]; ———— 前回の訪問ログを取り出します。$logdata に配列で入ります
07:     $counter = $logdata["counter"];
08:     $time = $logdata["time"];
09:     $lasttime = date("Y年n月j日 A g時i分", $time);
10: } else {
11:     // クッキーがないので訪問ログに初期値を設定する
12:     $counter = 0;
13:     $lasttime = "今回がはじめての訪問 ";
14: }
15: // 訪問ログをクッキーに保存する (24 時間有効) ———— 訪問回数を保存します
16: $result1 = setcookie('visitedLog[counter]', ++$counter, time()+60*60*24);
17: $result2 = setcookie('visitedLog[time]', time(), time()+60*60*24);
18: $result = ($result1 && $result2); ———— 訪問日時を保存します
19: ?>
20:
21: <!DOCTYPE html>
22: <html lang="ja">
23: <head>
24: <meta charset="utf-8">
25: <title>Page 1</title>
26: <link href="../../css/style.css" rel="stylesheet">
27: </head>
28: <body>
29: <div>
30:     <?php
31:     if ($result) {
32:         echo " このページの訪問は ", es($counter), " 回目です。", "<br>";
33:         echo " 前回の訪問：", es($lasttime), "<hr>";
34:         echo '<a href="page2.php">ページを移動する</a>', "<br>";
35:         echo ' (<a href="reset_log.php">リセットする</a>) ';
36:     } else {
37:         echo '<span class="error">クッキーが利用できませんでした。</span>';
38:     }
39:     ?>
40: </div>
41: </body>
42: </html>

```

配列の値をクッキーに保存する

クッキーへの保存は配列の要素ごとに行います。訪問回数の \$counter は保存する際に ++\$counter のようにカウントアップした後で同時に保存します。画面に表示する際にはカウントアップ済みの \$counter の値が表示されます。

訪問日時は time() の値を保存します。2 個のクッキーの保存結果を合わせて成功不成功を判断したいので、それぞれの結果の論理積の (\$result1 && \$result2) を \$result に入力します。

php 配列の値をクッキーに保存する

«sample» visited_log/page1.php

```
15: // 訪問ログをクッキーに保存する (24 時間有効) 配列 [ キー ] を文字列で指定します
16: $result1 = setcookie('visitedLog[counter]', ++$counter, time()+60*60*24);
17: $result2 = setcookie('visitedLog[time]', time(), time()+60*60*24);
18: $result = ($result1 && $result2);  クッキーに保存する訪問日時
```

クッキーから配列の値を取り出す

先にも書いたようにクッキーから配列の値を取り出すには、まず \$_COOKIE["visitedLog"] のように配列名で配列を取り出し、そこから \$logdata["counter"]、\$logdata["time"] のように各値を取り出します。日時は Unix タイムスタンプの秒数なので、date() を使って日付フォーマットを指定して「年月日 AM/PM 時分」の形式の文字列にしています。

\$_COOKIE["visitedLog"] に値がなかった場合は、はじめて訪問したかクッキーが破棄されてしまった場合です。その場合は変数の \$counter と \$lasttime に初期値を設定します。

php クッキーから配列の値を取り出す

«sample» visited_log/page1.php

```
04: if (isset($_COOKIE["visitedLog"])){
05:     // 訪問ログの値を取り出す
06:     $logdata = $_COOKIE["visitedLog"]; 訪問ログは $logdata に配列で入ります
07:     $counter = $logdata["counter"];
08:     $time = $logdata["time"]; 値は配列からキーで取り出します
09:     $lasttime = date("Y年n月j日 A g時i分", $time); フォーマット文字を使って秒数の日付データを
10: } else { 読みやすくします
11:     // クッキーがないので訪問ログに初期値を設定する
12:     $counter = 0;
13:     $lasttime = " 今回がはじめての訪問 ";
14: }
```

日付のフォーマット

クッキーの有効時間の設定で説明したように、訪問日時 `time()` の値は秒数です (P.439)。これを読みやすい年月日の表記にするために `date()` の日付フォーマットの機能を利用します。日付フォーマットは `date()` だけでなく、`DateTime` クラスの `DateTime::format()`、`date_format()` などでも同じように指定します。

よく利用するフォーマット文字は次のとおりです。フォーマット文字で指定すると、その文字が日時の実際の値と置き換わります。

指定文字	値の説明	実際の値
Y	年 (4 桁)	例 2001、2016、2021
y	年 (2 桁)	例 01、16、21
m	月	01 から 12 (ゼロを付ける)
n	月	1 から 12
M	月 (3 文字形式)	Jan から Dec
F	月	January から December
d	日	01 から 31 (ゼロを付ける)
j	日	1 から 31
D	曜日 (3 文字形式)	Mon から Sun
l (小文字の L)	曜日	Monday から Sunday
w	曜日	0 (日曜) から 6 (土曜)
a	午前／午後	am または pm
A	午前／午後	AM または PM
g	時 (12 時制)	1 から 12
G	時 (24 時制)	0 から 23
h	時 (12 時制)	01 から 12 (ゼロを付ける)
H	時 (24 時制)	00 から 23 (ゼロを付ける)
i	分	00 から 59 (ゼロを付ける)
s	秒	00 から 59 (ゼロを付ける)
I (大文字の i)	サマータイム中かどうか	1 (サマータイム中)、0 (サマータイム中ではない)
z	年間の通算日	0 から 365

NOTE

タイムゾーンの設定
日時がずれている場合は `php.ini` の `date.timezone` が "Asia/Tokyo" に指定してあるかどうか確認してください。(P.26、P.38、P.47)

訪問ログのクッキーを破棄する

訪問ログのクッキーを破棄する場合には、配列の値を保存したときと同じように値を1個ずつ破棄する必要があります。クッキーを破棄する方法は通常と同じで、`setcookie()` で有効期限を過去にして値を設定します。



php 訪問ログのクッキーを破棄する

«sample» visited_log/reset_log.php

```
01: <?php
02: // クッキーを破棄する
03: $result1 = setcookie('visitedLog[counter]', "", time()-3600); // count クッキーの破棄
04: $result2 = setcookie('visitedLog[time]', "", time()-3600); // time クッキーの破棄
05: $result = ($result1 && $result2);
06: ?>
07:
08: <!DOCTYPE html>
09: <html lang="ja">
10: <head>
11: <meta charset="utf-8">
12: <title>リセットページ</title>
13: <link href="../../../css/style.css" rel="stylesheet">
14: </head>
15: <body>
16: <div>
17: <?php
18:     if ($result){
19:         echo " 訪問ログのクッキーを破棄しました。", "<hr>";
20:         echo '<a href="page1.php">Page 1に戻る </a>';
21:     } else {
22:         echo '<span class="error"> クッキーの破棄でエラーがありました。</span>';
23:     }
24: ?>
25: </div>
26: </body>
27: </html>
```

有効期限を過去にします

Section 10-6

複数の値を 1 つにまとめてクッキーに保存する

前節でクッキーに配列を保存する方法を説明しましたが、クッキーに保存できる個数には制限があるので、値の個数が多い場合は複数の値を 1 つにまとめて 1 個のクッキーに保存するほうがよいでしょう。この節ではインデックス配列の場合と連想配列の場合に分けて、配列を文字列に変換してクッキーに保存する方法を紹介します。

インデックス配列を 1 個の文字列にして保存する

配列の値を連結して 1 個の文字列にし、それをクッキーに保存します。配列の値は `implode()` で簡単に文字列に連結できます。値と値の間を連結する文字は何でもかまいませんが、この例では「値 1 & 値 2 & 値 3」のように "&" で値を連結します。配列 `$fruits` に入っている ["りんご", "みかん", "レモン", "バナナ"] は、`implode("&", $fruits)` によって "りんご & みかん & レモン & バナナ" の文字列に変換できます。



php 配列の値を文字列に連結してクッキーに保存する

«sample»array_string/set_cookie.php

```

01: <?php
02: require_once("../lib/util.php");
03: // 保存する配列
04: $fruits = ["りんご", "みかん", "レモン", "バナナ"];
05: // 値を連結した文字列にする
06: $valueString = implode("&", $fruits); // 保存する値を1個の文字列に連結します
07: // クッキーに保存する
08: $result = setcookie("fruits", $valueString); // クッキーに保存します
09: ?>
10:
11: <!DOCTYPE html>
12: <html lang="ja">
13: <head>
14: <meta charset="utf-8">
15: <title>クッキーを保存する</title>

```

```

16: <link href="../../../css/style.css" rel="stylesheet">
17: </head>
18: <body>
19: <div>
20:     <?php
21:     if ($result) {
22:         echo "好きなフルーツを保存しました。", "<hr>";
23:         echo '<a href="check_cookie.php">クッキーを確認する </a>';
24:     } else {
25:         echo '<span class="error">クッキーが利用できませんでした。</span>';
26:     }
27:     ?>
28: </div>
29: </body>
30: </html>

```

クッキーから取り出した文字列を配列に戻す

クッキーに保存した値は配列の値を連結した文字列なので、`explode()` を使って元の配列に戻します。"&" で連結したので、`explode("&", $valueString)` で配列に変換できます。このサンプルでは配列の値を再び `implode("<br>", $fruits)` を使い改行タグ `<br>` で連結された文字列に変換して表示しています (P.207)。

`$valueString = $_COOKIE["fruits"]`

"りんご & みかん & レモン & バナナ"

`$fruits = explode("&", $valueString)`

["りんご", "みかん", "レモン", "バナナ"]

`implode("<br>", $fruits)`

"りんご
 みかん
 レモン
 バナナ"

php クッキーから取り出した文字列を配列に戻す

«sample» array_string/check_cookie.php

```

01: <?php
02: require_once("../lib/util.php");
03: ?>
04:
05: <!DOCTYPE html>
06: <html lang="ja">
07: <head>
08:     <meta charset="utf-8">
09:     <title>クッキーを確認する</title>
10:     <link href="../../../css/style.css" rel="stylesheet">
11: </head>
12: <body>
13: <div>
14:     <?php
15:     // クッキーの値を取り出す
16:     if (isset($_COOKIE["fruits"])){
17:         // 訪問ログの値を取り出す
18:         $valueString = $_COOKIE["fruits"]; ——— クッキーの値を取り出します

```

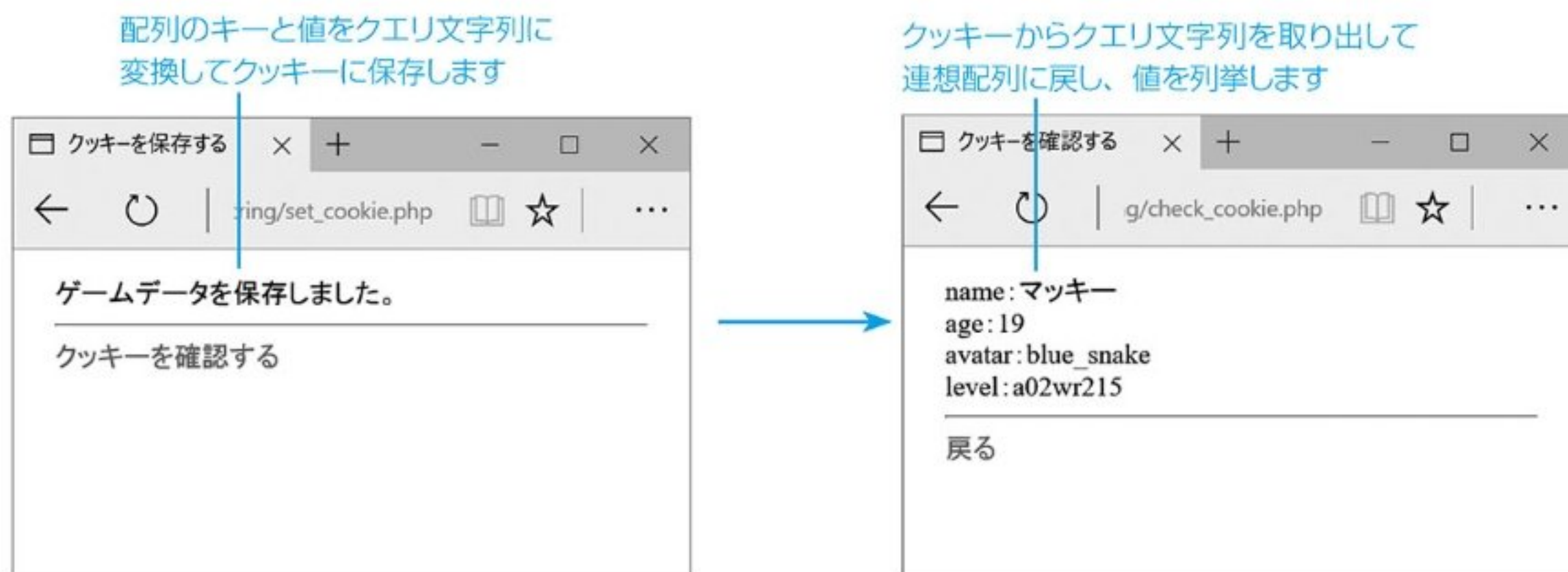
```

19: // 値を配列にする
20: $fruits = explode("&", $valueString); // & で連結しておいた文字列を配列に分割します
21: // HTML エスケープする
22: $fruits = es($fruits);
23: // 配列の値を列挙する
24: echo "好きなフルーツ:", "<br>";
25: echo implode("<br>", $fruits), "<hr>"; // ブラウザに表示するために <br> で
26: } else {                               連結した文字列を作ります
27:     echo "クッキーはありません。", "<hr>";
28: }
29: ?>
30: <a href="set_cookie.php">戻る </a>
31: </div>
32: </body>
33: </html>

```

連想配列を 1 個のクエリ文字列にして保存する

連想配列をクッキーに保存したい場合は、「キー 1= 値 1& キー 2= 値 2」のようにクエリ文字列に変換して保存する方法があります。クエリ文字列で保存すれば、`parse_str()` を利用して連想配列に戻すことができます。連想配列をクエリ文字列に変換する関数はないので、`array_queryString()` をユーザ定義します。



php 連想配列のキーと値をクエリ文字列にして保存する

«sample» array_querystring/set_cookie.php

```

01: <?php
02: require_once("../lib/util.php");
03: // 保存する連想配列
04: $gamedata = ["name"=>"マッキー", "age"=>19, "avatar"=>"blue_snake", "level"=>"a02wr215"];
05: // 連想配列をクエリ文字列にする
06: $dataQueryString = array_queryString($gamedata);
07: // クッキーに保存する ()
08: $result = setcookie("gamedata", $dataQueryString, time()+60*5);
09: ?>
10:
11: <?php
12: // 連想配列のキーと値をクエリ文字列に変換する
13: function array_queryString(array $variable): string {
14:     $data = [];

```

連想配列をクエリ文字列に変換する関数を定義します

```

15:     foreach ($variable as $key => $value) {
16:         $data[] = "{$key}={$value}";
17:     }
18:     // クエリ文字列を作る
19:     $queryString = implode("&", $data);
20:     return $queryString;
21: }
22: ?>
23:
24: <!DOCTYPE html>
25: <html lang="ja">
26: <head>
27: <meta charset="utf-8">
28: <title>クッキーを保存する</title>
29: <link href="../../css/style.css" rel="stylesheet">
30: </head>
31: <body>
32: <div>
33:     <?php
34:     if ($result) {
35:         echo "ゲームデータを保存しました。", "<hr>";
36:         echo '<a href="check_cookie.php">クッキーを確認する</a>';
37:     } else {
38:         echo '<span class="error">クッキーが利用できませんでした。</span>';
39:     }
40:     ?>
41: </div>
42: </body>
43: </html>

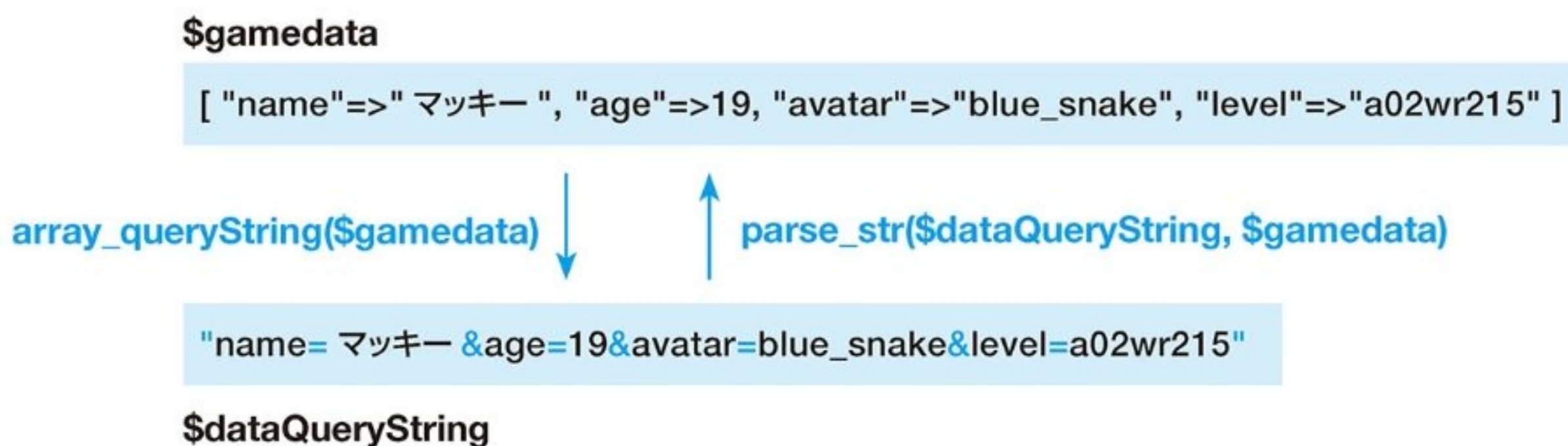
```

"キー = 値" の要素にして配列に追加します

できた配列を "&" で連結してクエリ文字列にします

連想配列をクエリ文字列に変換する関数を定義する

連想配列を「キー 1= 値 1& キー 2= 値 2」の形式のクエリ文字列に変換する関数を定義しておくると便利です。引数で連想配列を受け取ったならば、foreach 文を使ってキーと値を順に取り出して["キー 1= 値 1", "キー 2= 値 2"]の形式の配列に変換し (foreach ➡ P.219)、それをあらかじめ implode() を使って "&" で連結したクエリ文字列に変換します。



php 連想配列のキーと値をクエリ文字列に変換する

«sample» array_querystring/set_cookie.php

```

12: // 連想配列のキーと値をクエリ文字列に変換する
13: function array_queryString(array $variable): string {
14:     $data = [];
15:     foreach ($variable as $key => $value) {
16:         $data[] = "{$key}={$value}"; // "キー = 値" の1個の文字列にして配列に追加します
17:     }
18:     // クエリ文字列を作る
19:     $queryString = implode("&", $data); // できあがった配列を & で連結してクエリ文字列にします
20:     return $queryString;
21: }

```

クッキーから取り出したクエリ文字列を連想配列に戻す

クッキーから取り出した値はクエリ文字列なので、`parse_str()` を使って連想配列に戻します。配列のままで HTML エスケープし (P.306)、`foreach` 文を使ってすべてのキーと値のペアをブラウザに表示します。

php クッキーから取り出したクエリ文字列を配列に戻す

«sample» array_querystring/check_cookie.php

```

01: <?php
02: require_once("../lib/util.php");
03: ?>
04:
05: <!DOCTYPE html>
06: <html lang="ja">
07: <head>
08:     <meta charset="utf-8">
09:     <title>クッキーを確認する</title>
10:     <link href="../../css/style.css" rel="stylesheet">
11: </head>
12: <body>
13: <div>
14:     <?php
15:     // クッキーの値を取り出す
16:     if (isset($_COOKIE["gamedata"])){
17:         // ゲームデータの値を取り出す
18:         $dataQueryString = $_COOKIE["gamedata"]; // クッキーの値を取り出します
19:         // クエリ文字列から連想配列を作る
20:         parse_str($dataQueryString, $gamedata); // 値のクエリ文字列を連想配列に戻します
21:         // HTML エスケープ
22:         $gamedata = es($gamedata);
23:         // 連想配列の値を列挙する
24:         foreach ($gamedata as $key => $value) {
25:             echo "{$key} : {$value}", "<br>"; // クッキーに保存していたキーと値を表示します
26:         }
27:         echo "<hr>";
28:     } else {
29:         echo "クッキーはありません。", "<hr>";
30:     }
31: ?>
32: <a href="set_cookie.php">戻る</a>
33: </div>
34: </body>
35: </html>

```